LAPORAN PROYEK 1

INTELLIGENT MULTI-AGENT SYSTEM PROTOTYPE

T2D-MAS: Ticket-to-Deployment Multi-Agent System

Sistem Multiagen Cerdas untuk Otomasi Siklus Pengiriman Perangkat Lunak pada Perusahaan Digital

Mata Kuliah: Agen Cerdas Enterprise

Dosen Pengampu: Prof. Dr. Drs. Azhari, M.T.

Kelompok: Agentic Delivery Lab

Disusun oleh:

Muhammad Zidan Arsyad (25/572381/PPA/07192)

Alissa Dyah Ayu Permatasari (25/575323/PPA/07279)

Muhammad Fikry Rizal (25/573936/PPA/07229)

Elsa Aiziyah (25/572243/PPA/07187)

Farichaturrifqi Aryanitasari (25/574224/PPA/07240)

PROGRAM STUDI MAGISTER KECERDASAN ARTIFISIAL

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas terselesaikannya laporan Proyek 1 pada mata kuliah Agen Cerdas Enterprise yang berjudul *T2D-MAS: Ticket-to-Deployment Multi-Agent System*. Laporan ini memaparkan rancangan dan purwarupa sistem multiagen cerdas untuk mengotomasi sebagian besar siklus pengiriman perangkat lunak pada perusahaan digital, mulai dari penerimaan tiket keluhan pengguna sampai perbaikan tayang di lingkungan produksi.

Laporan ini disusun mengikuti sepuluh butir tata letak tugas Proyek 1, yaitu identitas kelompok, pernyataan masalah, kajian *state of the art*, tujuan proyek, desain sistem multiagen, algoritma dan integrasi kecerdasan artifisial, basis data dan *dataset*, purwarupa dan fitur, evaluasi, serta kesimpulan dan rekomendasi. Isi laporan merupakan pengembangan dari Laporan Progres yang berjudul *From Ticket to Deployment: Mapping the Software Delivery Lifecycle Across 8 Eras of AI Capability and Autonomy*.

Kami mengucapkan terima kasih kepada dosen pengampu yang telah membimbing kami selama pengerjaan proyek ini. Kami menyadari bahwa laporan ini masih memiliki keterbatasan. Oleh karena itu, kami mengharapkan kritik dan saran yang membangun demi penyempurnaan pekerjaan berikutnya. Semoga laporan ini bermanfaat bagi pembaca.

Yogyakarta, September 2026

Tim Penyusun

ABSTRAK

Proses pengiriman perangkat lunak pada perusahaan digital masih bergantung pada intervensi manual di hampir setiap transisi antartahap. Ketergantungan ini menimbulkan penyelesaian tiket yang lambat, kerja ulang akibat tiket duplikat, antrean persetujuan, mutu tiket yang tidak seragam, biaya observabilitas terpusat, dan percepatan tanpa pengaman. Laporan ini merancang dan memprototipekan T2D-MAS, sebuah sistem multiagen cerdas yang mengoordinasikan agen *Scout*, *Broker*, *Worker*, dan *Security* sepanjang sembilan tahap alur kerja sambil mempertahankan tiga gerbang persetujuan manusia.

Rancangan sistem memadukan aturan, *machine learning*, *deep learning*, *reinforcement learning*, *graph neural network*, dan model bahasa besar, yang masing-masing ditempatkan pada tahap yang sesuai. Selain itu, sistem membedakan agen statis dan agen bergerak (*mobile agent*) dengan aturan migrasi berbasis perbandingan volume data terhadap ukuran agen, serta mengendalikan otonomi melalui aturan gerbang berbasis tingkat keyakinan dan risiko.

Evaluasi membandingkan empat konfigurasi, yaitu A0 sampai A3, pada 500 tiket sintetis untuk setiap konfigurasi. Hasilnya menunjukkan bahwa semua konfigurasi mencapai *macro-F1* 1,00 pada tugas sintetis tersebut, konfigurasi A1 sampai A3 memprioritaskan kesalahan lebih baik daripada A0 (NAPFD 0,75 berbanding 0,49), dan konfigurasi bergerak (A3) menambah biaya komunikasi tanpa peningkatan kualitas yang jelas pada beban data kecil. Hasil ini merupakan hasil kelas pada data sintetis dan bukan bukti manfaat di lapangan.

Kata kunci: sistem multiagen, agen bergerak, triase tiket, *Contract Net Protocol*, tata kelola otonomi, *ticket-to-deployment*

DAFTAR ISI

KATA PENGANTAR i
ABSTRAK ii
DAFTAR ISI iii
DAFTAR TABEL v
DAFTAR GAMBAR vii
BAB 1 IDENTITAS KELOMPOK 1
1.1 Identitas 1
1.2 Anggota dan Pembagian Peran 1
1.3 Matriks Kontribusi (RACI Ringkas) 1
1.4 Kanal Kolaborasi 2
BAB 2 PERNYATAAN MASALAH 3
2.1 Konteks Perusahaan 3
2.2 Deskripsi Masalah 3
2.3 Kuantifikasi Dampak (Baseline Ilustratif) 5
2.4 Pemetaan Pain Point ke Kebutuhan Sistem 6
BAB 3 STATE OF THE ART 8
3.1 Sistem dan Produk yang Sudah Ada di Dunia Nyata 8
3.2 Analisis Perbandingan 10
3.3 Celah yang Diisi Proyek Ini 12
3.4 Kelebihan dan Kekurangan Konsep yang Diusulkan 12
BAB 4 TUJUAN PROYEK 14
4.1 Tujuan Umum 14
4.2 Tujuan Khusus 14
4.3 Target Terukur (SMART) 14
4.4 Ruang Lingkup dan Batasan 15
BAB 5 DESAIN SISTEM MULTIAGEN 16
5.1 Spesifikasi Sistem dan Fitur 16
5.2 Justifikasi Agen Tunggal dan Multiagen 17
5.3 Taksonomi Agen: Scout, Broker, Worker, dan Security 18
5.4 Klasifikasi Agen Bergerak dan Agen Statis 20
5.5 Diagram Arsitektur Sistem 23
5.6 Diagram Koordinasi, Kolaborasi, dan Negosiasi (MIMA) 24
5.7 Batas Otonomi dan Tata Kelola (*Autonomy Boundary and Governance*) 26
BAB 6 ALGORITMA DAN INTEGRASI AI 27
6.1 Prinsip Pemilihan Metode 27
6.2 Berbasis Aturan: Validasi dan Gerbang Kepatuhan 27
6.3 Machine Learning Klasik: Klasifikasi Kategori dan Pengarahan 28
6.4 Deep Learning: Deteksi Duplikat dan Klasifikasi Severity 28
6.5 Reinforcement Learning 30
6.6 Graph Neural Network: Lokalisasi Akar Masalah 31
6.7 Generative AI dan LLM 32

6.8 Aturan Keputusan Otonomi (Gate Rule) 33
6.9 Pertimbangan Untung-Rugi Antarmetode 33
BAB 7 BASIS DATA DAN DATASET 35
7.1 Basis Data Operasional 35
7.2 Basis Data Vektor 37
7.3 Dataset untuk Pelatihan ML dan DL 38
7.4 Integrasi API Perusahaan (Simulasi) 41
7.5 Tata Kelola Data 42
BAB 8 PURWARUPA DAN FITUR 43
8.1 Lingkup Purwarupa 43
8.2 Tumpukan Teknologi 43
8.3 Rancangan UI/UX 44
8.4 Fitur Unggulan 46
8.5 Simulasi Kasus Nyata: Insiden Flash Sale 46
BAB 9 EVALUASI 48
9.1 Desain Eksperimen 48
9.2 Metrik 49
9.3 Rumus Perbandingan 50
9.4 Hasil Percobaan 50
9.5 Uji Statistik 52
9.6 Ancaman terhadap Validitas 53
BAB 10 KESIMPULAN DAN REKOMENDASI 54
10.1 Ringkasan Manfaat Sistem 54
10.2 Keterbatasan 55
10.3 Rekomendasi Pengembangan 55
10.4 Penutup 56
REFERENSI 58
TAUTAN KODE PROGRAM 61
12.1 Struktur Repositori 61
LAMPIRAN A STATUS KELENGKAPAN DOKUMEN 62
LAMPIRAN B SPESIFIKASI IMPLEMENTASI 63
B.1 Modul Triase (Deduplikasi dan Severity) 63
B.2 Lingkungan RL 64
B.3 Modul RCA Berbasis Graf 64
B.4 Lapisan Data 65
B.5 Kerangka Agen dan Orkestrator 65
B.6 Dasbor 66
B.7 Harness Evaluasi 67

DAFTAR TABEL

Tabel 1.	Anggota kelompok, peran, dan tanggung jawab	1
Tabel 2.	Matriks kontribusi (RACI)	2
Tabel 3.	Kanal kolaborasi	2
Tabel 4.	Parameter baseline (asumsi)	5
Tabel 5.	Perhitungan pemborosan bulanan	6
Tabel 6.	Penghematan teoretis bulanan	6
Tabel 7.	Pemetaan *pain point* ke kebutuhan sistem	7
Tabel 8.	Perbandingan cakupan, taksonomi, agen bergerak, dan tata kelola	11
Tabel 9.	Perbandingan integrasi RL, GNN, ketergantungan vendor, dan kematangan	11
Tabel 10.	Tujuan khusus proyek	14
Tabel 11.	Target terukur prototipe	15
Tabel 12.	Kebutuhan fungsional	16
Tabel 13.	Kebutuhan nonfungsional	17
Tabel 14.	Taksonomi agen dan instansiasinya	18
Tabel 15.	Komponen internal agen: masukan, memori, dan inti penalaran	19
Tabel 16.	Komponen internal agen: *tool* dan keluaran	20
Tabel 17.	Klasifikasi mobilitas agen	21
Tabel 18.	Perbandingan biaya komunikasi agen statis dan agen bergerak (asumsi)	22
Tabel 19.	Risiko mobilitas agen dan mitigasinya	23
Tabel 20.	Perhitungan utilitas tawaran pada tiket TCK-1042	25
Tabel 21.	Batas otonomi dan tata kelola	26
Tabel 22.	Pemetaan tahap alur ke metode	27
Tabel 23.	Vektor fitur pada contoh perhitungan kemiripan tiket	29
Tabel 24.	Contoh perhitungan *softmax* untuk satu tiket	30
Tabel 25.	Fitur simpul pada contoh GNN	32
Tabel 26.	Tugas LLM, pola *prompt*, dan pengamannya	33
Tabel 27.	Perbandingan karakteristik metode	34
Tabel 28.	Tabel basis data operasional	36
Tabel 29.	Rancangan basis data vektor	38
Tabel 30.	Dataset yang dipertimbangkan	39
Tabel 31.	Metrik dataset pada percobaan yang dilaporkan	41
Tabel 32.	Layanan tiruan dan titik akhir yang ditiru	41
Tabel 33.	Aturan tata kelola data	42
Tabel 34.	Lingkup dan status purwarupa	43
Tabel 35.	Tumpukan teknologi	44
Tabel 36.	Fitur unggulan dan cara demonstrasi	46

Tabel 37. Linimasa simulasi insiden *flash sale* 47
Tabel 38. Konfigurasi percobaan 48
Tabel 39. Definisi operasional indikator evaluasi 49
Tabel 40. Hasil percobaan pada 500 tiket sintetis per konfigurasi 51
Tabel 41. Target rancangan dan status pengukuran 52
Tabel 42. Uji statistik yang dirancang 52
Tabel 43. Ancaman terhadap validitas dan mitigasinya 53
Tabel 44. Rekomendasi pengembangan 56
Tabel 45. Artefak dan tautan proyek 61
Tabel 46. Butir yang perlu dilengkapi 62
Tabel 47. Urutan eksekusi spesifikasi implementasi 63

DAFTAR GAMBAR

Gambar 1.	Arsitektur sistem T2D-MAS	24
Gambar 2.	Diagram koordinasi dan negosiasi (MIMA)	25
Gambar 3.	Diagram relasi entitas basis data T2D-MAS	37
Gambar 4.	Kerangka (*wireframe*) dasbor T2D-MAS	44

BAB 1

IDENTITAS KELOMPOK

1.1 Identitas

Kelompok kami bernama Agentic Delivery Lab dan berasal dari kelas Agen Cerdas Enterprise. Kelompok ini mengerjakan Proyek 1 dengan tema sistem multiagen cerdas untuk otomasi siklus pengiriman perangkat lunak.

1.2 Anggota dan Pembagian Peran

Tata letak tugas meminta enam peran, yaitu *Leader*, *Researcher*, *Programmer*, *Designer*, *Evaluator*, dan *Presenter*, sedangkan kelompok kami beranggotakan lima orang. Oleh karena itu, satu anggota memegang peran ganda. Pembagian peran dan tanggung jawab setiap anggota tersaji pada Tabel 1.

Tabel 1. Anggota kelompok, peran, dan tanggung jawab

No.	Nama	NIM	Peran utama	Peran sekunder	Tanggung jawab konkret
1	Muhammad Zidan Arsyad	25/572381/PPA/07192	*Leader*	*Presenter*	Koordinasi jadwal, konsolidasi dokumen, Bab 1, 4, 10, dan 12, serta pemaparan proyek
2	Alissa Dyah Ayu Permatasari	25/575323/PPA/07279	*Researcher*	-	Studi literatur, kajian *state of the art*, validasi klaim, Bab 2, 3, dan 11
3	Muhammad Fikry Rizal	25/573936/PPA/07229	*Programmer*	-	Implementasi agen, API simulasi, model ML/DL/RL, Bab 6 dan 7
4	Elsa Aiziyah	25/572243/PPA/07187	*Designer*	-	Arsitektur dan diagram koordinasi, desain UI/UX dasbor, Bab 5 dan 8
5	Farichaturrifqi Aryanitasari	25/574224/PPA/07240	*Evaluator*	-	Desain eksperimen, metrik, analisis hasil, Bab 9

1.3 Matriks Kontribusi (RACI Ringkas)

Matriks kontribusi pada Tabel 2 menunjukkan siapa yang mengerjakan dan siapa yang bertanggung jawab akhir atas setiap aktivitas. Keterangan

singkatannya adalah sebagai berikut: R berarti mengerjakan, A berarti bertanggung jawab akhir, C berarti dikonsultasi, dan I berarti diinformasikan. Pada laporan progres sebelumnya, seluruh sel matriks diisi tanda centang untuk semua anggota. Format RACI ini kami pilih karena menunjukkan akuntabilitas yang dapat ditelusuri, dengan satu penanggung jawab akhir (A) pada setiap baris.

Tabel 2. Matriks kontribusi (RACI)

Aktivitas	Zidan	Alissa	Fikry	Elsa	Icha
Perumusan masalah dan studi literatur	A	R	C	C	C
Ilustrasi data dan perhitungan komputasi	I	C	R	C	A
Desain arsitektur sistem multiagen	A	C	C	R	I
Implementasi metode AI/ML/DL/RL	I	C	R/A	I	C
Desain UI/UX dan purwarupa	C	I	C	R/A	C
Desain evaluasi dan analisis hasil	C	C	C	I	R/A
Penyusunan dan penyuntingan laporan	R/A	R	R	R	R

1.4 Kanal Kolaborasi

Kelompok kami menggunakan empat kanal kolaborasi, seperti tersaji pada Tabel 3. Repositori kode dikelola di GitHub. Manajemen tugas memakai GitHub Projects atau Jira versi gratis, yang sekaligus menjadi sumber data tiket simulasi (lihat Bab 7). Dokumen disusun di Google Docs dan diekspor ke PDF, dengan versi final yang dikunci satu hari sebelum pengumpulan. Adapun diagram disimpan sebagai teks Mermaid dan draw.io agar dapat dikelola dengan sistem kendali versi.

Tabel 3. Kanal kolaborasi

Kebutuhan	Perangkat	Keterangan
Repositori kode	GitHub	Tautan repositori tercantum pada Bab 12
Manajemen tugas	GitHub Projects / Jira versi gratis	Sekaligus menjadi sumber data tiket simulasi (Bab 7)
Dokumen	Google Docs, diekspor ke PDF	Versi final dikunci satu hari sebelum pengumpulan
Diagram	Mermaid dan draw.io	Sumber diagram disimpan sebagai teks agar dapat dikelola dengan sistem kendali versi

BAB 2

PERNYATAAN MASALAH

2.1 Konteks Perusahaan

Studi kasus proyek ini diambil pada sebuah perusahaan *e-commerce* nasional berskala menengah yang mengoperasikan aplikasi seluler dan web dengan arsitektur *microservices*. Divisi Engineering perusahaan tersebut beranggotakan sekitar 45 orang dalam 7 skuad, dan bertanggung jawab atas seluruh siklus kerja sejak keluhan pengguna diterima hingga perbaikan tayang di produksi.

Kami memilih domain ini karena memenuhi tiga syarat tugas sekaligus. Pertama, domain ini merupakan masalah nyata pada perusahaan bisnis. Kedua, domain ini memiliki volume transaksi dan data yang cukup untuk melatih model. Ketiga, domain ini memiliki koordinasi lintas unit, yaitu *Support*, QA, *Dev*, SRE, dan *Security*, yang secara alami memerlukan banyak agen, bukan satu agen tunggal.

Siklus kerja yang berlaku saat ini kami petakan menjadi sebuah alur sembilan tahap. Urutannya adalah (1) *Feedback Capture*, (2) *Smart Triage*, (3) *Investigation*, (4) *Planning*, (5) *Implementation*, (6) *QA/Verification*, (7) *Deployment*, (8) *Monitoring*, dan (9) *Feedback Loop*.

2.2 Deskripsi Masalah

Hampir setiap transisi antartahap pada alur di atas masih bergantung pada intervensi manual. Akibatnya, proses tidak hanya lambat, tetapi juga tidak konsisten dan mahal. Kami mengidentifikasi enam titik masalah (*pain point*) yang diuraikan berikut ini.

2.2.1 Pain Point 1: Waktu Penyelesaian yang Panjang akibat Triase Manual

Klasifikasi kategori, penentuan tingkat keparahan (*severity*), dan pengarahan ke skuad dilakukan oleh *support engineer* berdasarkan penilaian subjektif dan pencarian kata kunci yang tidak presisi. Dua tiket dengan makna

yang sama tetapi diksi berbeda, misalnya "aplikasi *force-close*" dan "*app crash*", diperlakukan sebagai dua masalah yang berbeda. Kondisi ini merupakan karakteristik sistem Era 1 dan 2, yaitu berbasis aturan dan pencocokan kata kunci.

2.2.2 Pain Point 2: Kerja Ulang akibat Tiket Duplikat

Tiket yang pernah dilaporkan tidak terdeteksi karena pencocokan hanya berbasis kata kunci yang persis sama. Akibatnya, tim mengeluarkan usaha investigasi berulang untuk akar masalah yang identik.

Besaran masalah ini bukan spekulasi. Studi pada repositori Bugzilla periode 2017 sampai 2022 melaporkan rasio duplikat sebesar 6,60 persen pada Eclipse, 20,53 persen pada Firefox, dan 10,56 persen pada proyek *Mobile*. Artinya, pada proyek dengan basis pengguna awam yang besar, satu dari lima laporan berpotensi merupakan pengulangan.

2.2.3 Pain Point 3: Hambatan Manusia pada Gerbang Persetujuan

Persetujuan manusia pada awal *sprint*, penggabungan *pull request*, dan *deployment* memang diperlukan untuk mengendalikan risiko. Masalah muncul ketika persetujuan itu tidak didukung ringkasan atau analisis otomatis. Pemberi persetujuan terpaksa membaca konteks mentah sehingga antrean menumpuk pada jam sibuk.

Laporan DORA 2025 memberikan konteks industri yang relevan. Laporan tersebut menunjukkan bahwa hanya 9,4 persen tim yang mencapai *lead time* di bawah satu jam, sedangkan 43,5 persen tim membutuhkan lebih dari satu minggu. Selain itu, 23,9 persen organisasi melakukan *deployment* kurang dari sekali dalam sebulan.

2.2.4 Pain Point 4: Kontrol Mutu yang Tidak Konsisten

Kriteria penerimaan (*acceptance criteria*), langkah reproduksi, dan catatan rilis (*release notes*) ditulis secara manual sehingga kelengkapannya bergantung pada penulisnya. Tiket dengan deskripsi yang buruk memaksa adanya siklus klarifikasi tambahan yang tidak tercatat sebagai pekerjaan.

2.2.5 Pain Point 5: Biaya Observabilitas Terpusat

Investigasi insiden mengharuskan penarikan log mentah dari banyak simpul produksi ke mesin analis. Untuk satu jendela insiden, volume log dapat mencapai satuan gigabita per simpul. Pola "memindahkan data ke komputasi" ini membebani jaringan, memperlambat waktu pemulihan rata-rata (MTTR), dan menimbulkan risiko kepatuhan karena data mentah yang berpotensi mengandung data pribadi (PII) keluar dari batas lingkungan produksi. Titik masalah inilah yang menjadi justifikasi utama penggunaan agen bergerak (Subbab 5.4).

2.2.6 Pain Point 6: Percepatan tanpa Pengaman

Adopsi perangkat pengodean berbasis AI mempercepat keluaran individu. Namun, riset industri 2025 sampai 2026 mencatat efek samping berupa peningkatan laju kegagalan perubahan (*change failure rate*), ukuran *pull request*, dan beban tinjauan yang kerap menyertai peningkatan *throughput*. Oleh karena itu, sistem yang kami rancang tidak boleh hanya mengejar kecepatan. Sistem ini harus membawa gerbang mutu dan keamanan sebagai komponen utama.

2.3 Kuantifikasi Dampak (Baseline Ilustratif)

Angka pada subbab ini merupakan asumsi terkalibrasi untuk perusahaan berukuran sedang dan dapat diganti dengan data internal apabila tersedia. Parameter yang digunakan tersaji pada Tabel 4.

Tabel 4. Parameter baseline (asumsi)

Parameter	Simbol	Nilai (asumsi)	Sumber atau dasar
Volume tiket masuk per bulan	N	1.200 tiket	Asumsi operasional
Waktu triase manual per tiket	t_1	12 menit	Asumsi, wawancara *support*
Rasio duplikat	d	15 persen	Interpolasi Eclipse 6,6 persen sampai Firefox 20,5 persen
Waktu investigasi rata-rata per tiket duplikat	t_2	2,5 jam	Asumsi
Biaya *engineer* per jam	c	Rp150.000	Asumsi

Perhitungan pemborosan bulanan dilakukan dengan lima rumus berikut. Beban triase adalah hasil kali volume tiket dengan waktu triase per tiket.

$$B_{triase} = N \times t_1 \quad (1)$$

Jumlah tiket duplikat adalah hasil kali volume tiket dengan rasio duplikat.

$$T_{dup} = N \times d \quad (2)$$

Beban kerja ulang adalah hasil kali jumlah tiket duplikat dengan waktu investigasi per tiket duplikat.

$$B_{rework} = T_{dup} \times t_2 \quad (3)$$

Total beban adalah jumlah beban triase dan beban kerja ulang, dan biaya ekuivalen adalah hasil kali total beban dengan biaya per jam.

$$B_{total} = B_{triase} + B_{rework} \quad (4)$$

$$Biaya = B_{total} \times c \quad (5)$$

Hasil perhitungan dengan parameter pada Tabel 4 tersaji pada Tabel 5. Total beban sebesar 690 jam per bulan setara dengan Rp103.500.000 per bulan atau sekitar Rp1,24 miliar per tahun.

Tabel 5. Perhitungan pemborosan bulanan

Komponen	Perhitungan	Hasil
Beban triase	1.200 × 12 menit	240 jam per bulan
Tiket duplikat	1.200 × 0,15	180 tiket per bulan
Beban kerja ulang	180 × 2,5 jam	450 jam per bulan
Total beban	240 + 450	690 jam per bulan
Biaya ekuivalen	690 × Rp150.000	Rp103.500.000 per bulan (sekitar Rp1,24 miliar per tahun)

Apabila sistem multiagen mampu mengotomasi 70 persen beban triase dan mendeteksi 85 persen duplikat sebelum investigasi dimulai, penghematan teoretisnya tersaji pada Tabel 6. Total penghematan sebesar 550,5 jam per bulan setara dengan sekitar Rp82,6 juta per bulan atau sekitar 80 persen dari pemborosan.

Tabel 6. Penghematan teoretis bulanan

Komponen	Perhitungan	Hasil
Hemat triase	240 × 0,70	168 jam

Komponen	Perhitungan	Hasil
Hemat kerja ulang	450 × 0,85	382,5 jam
Total penghematan	168 + 382,5	550,5 jam per bulan (sekitar Rp82,6 juta per bulan)

Angka penghematan inilah yang menjadi target yang diuji pada Bab 9. Perlu ditegaskan bahwa penghematan tersebut bersifat potensial. Penghematan hanya terealisasi jika tingkat kesalahan agen, seperti salah klasifikasi dan duplikat palsu, tetap berada di bawah ambang yang ditetapkan pada tabel *Autonomy Boundary* (Subbab 5.7).

2.4 Pemetaan Pain Point ke Kebutuhan Sistem

Setiap titik masalah dipetakan ke kebutuhan sistem, agen penanggung jawab, dan subbab yang membahasnya, seperti tersaji pada Tabel 7.

Tabel 7. Pemetaan *pain point* ke kebutuhan sistem

Pain point	Kebutuhan sistem	Agen penanggung jawab	Subbab terkait
1. Triase manual	Klasifikasi kategori dan *severity* otomatis	*Broker-Triage*	6.3, 6.4
2. Duplikat	Pencocokan semantik, bukan kata kunci	*Broker-Triage* dan *Vector DB*	6.4, 7.2
3. Hambatan persetujuan	Ringkasan keputusan, skor risiko, dan aturan gerbang	*Security-Policy*	5.7, 6.8
4. Mutu tidak konsisten	Generasi terstruktur (*acceptance criteria*, langkah reproduksi, catatan rilis)	*Worker-Plan*, *Worker-Impl*	6.7
5. Biaya observabilitas	Komputasi di lokasi data (agen bergerak)	*Scout-Log* (mobile)	5.4
6. Percepatan tanpa pengaman	Gerbang mutu berlapis dan metrik stabilitas	*Security* dan *Scout-Monitor*	9.2

BAB 3

STATE OF THE ART

3.1 Sistem dan Produk yang Sudah Ada di Dunia Nyata

Subbab ini mengkaji tujuh kelompok sistem yang relevan, yaitu lima produk komersial, satu pasangan agen pengodean, dan sistem akademik. Fitur produk pada subbab ini bersumber dari dokumentasi vendor dan perlu diverifikasi ulang pada tanggal pengumpulan karena peta jalan vendor berubah dengan cepat.

3.1.1 Atlassian Rovo (Jira dan Jira Service Management)

Rovo memosisikan agen AI sebagai "pekerja yang dapat dialamatkan" dan dapat dipanggil dari aturan otomasi. Pada sisi layanan, tersedia agen bawaan *Service Triage* yang menganalisis isi, sentimen, dan detail permintaan untuk menentukan jenis permintaan, tingkat urgensi, dan prioritas. Agen ini juga dapat menulis ulang judul atau deskripsi tiket secara langsung ketika tiket masuk ke antrean, dan dapat menentukan perlu tidaknya eskalasi berdasarkan SLA dan interaksi pelanggan.

Pada sisi rekayasa, Rovo Dev menyediakan empat kemampuan. *Code planning* menyusun rencana implementasi dari *work item* Jira dengan konteks spesifikasi Confluence dan *pull request* terdahulu. *Code generation* mengubah rencana menjadi draf *branch* dan *pull request*. *Code review* memvalidasi *pull request* terhadap kriteria penerimaan. *Code automation* menjalankan alur multilangkah secara paralel. Atlassian juga melaporkan bahwa otomasi agentik pada JSM tumbuh 3,4 kali lebih cepat daripada otomasi tradisional. Salah satu contoh penerapannya adalah agen yang menangani setiap tiket baru dengan sub-agen paralel untuk menghasilkan ringkasan, tiket serupa, dan rekomendasi.

3.1.2 ServiceNow AI Agents (Now Assist, AI Agent Studio, dan AI Agent Orchestrator)

ServiceNow menyediakan tiga komponen inti. *AI Agent Studio* digunakan untuk membangun agen kustom, *AI Agent Orchestrator* mengoordinasikan banyak

agen pada tugas kompleks, dan *AI Control Tower* mengelola tata kelola serta penegakan kebijakan tingkat perusahaan.

Pola penggunaan kanonik pada ITSM adalah sebagai berikut: agen mendeteksi masalah, mengumpulkan data dari sistem pemantauan, menilai dampak bisnis, mengidentifikasi akar masalah, menyusun rencana remediasi, mengeksekusi perbaikan setelah persetujuan manusia, lalu memperbarui dokumentasi. Praktik terbaik yang direkomendasikan komunitasnya sejalan dengan rancangan proyek ini, yaitu menerapkan *human-in-the-loop* untuk memvalidasi ringkasan dan resolusi agar terhindar dari halusinasi, serta peluncuran bertahap (*phased rollout*) dari area berisiko rendah.

3.1.3 IBM watsonx Orchestrate

Produk ini berperan sebagai *supervisor*, *router*, dan *planner* multiagen lintas tumpukan teknologi perusahaan. Produk ini menyediakan katalog agen siap pakai, pembangun agen *no-code* dan *pro-code*, serta tata kelola terpusat. Hal yang relevan secara arsitektural bagi proyek ini adalah penggunaan protokol standar untuk interoperabilitas, yaitu *Model Context Protocol* (MCP) untuk integrasi *tool* dan *Agent Communication/Collaboration Protocol* untuk komunikasi antaragen. Hal ini menjadi preseden bahwa pengiriman pesan antaragen sebaiknya memakai protokol terbuka, bukan pemanggilan fungsi ad hoc.

3.1.4 SAP Joule (Intelligent Enterprise)

SAP menyematkan agen AI di dalam asisten Joule yang berlandaskan data bisnis organisasi dan membantu secara proaktif pada aplikasi *cloud* yang digunakan pegawai. Kekuatannya terletak pada kedalaman integrasi ke proses ERP. Kelemahannya untuk kasus kami adalah cakupannya berpusat pada proses bisnis inti, bukan siklus rekayasa perangkat lunak.

3.1.5 GitHub Copilot Coding Agent

Copilot *coding agent* merupakan agen pengembang asinkron yang tersedia secara umum sejak akhir 2025 dan semakin matang pada 2026. Alur kerjanya diawali dengan pendelegasian tugas, yaitu dengan menetapkan isu kepada Copilot, memakai panel agen, atau menekan tombol delegasi di VS Code.

Selanjutnya, agen membuka draf *pull request* dan bekerja di lingkungan pengembangan efemeral yang ditenagai GitHub Actions. Di lingkungan itu, agen menjelajahi kode, membuat *branch*, menulis perubahan, menjalankan tes dan *linter*, mendeteksi kesalahan dan mengoreksi diri, lalu meminta tinjauan manusia. Tugas yang dicakup meliputi implementasi fitur, perbaikan *bug*, penanganan utang teknis, peningkatan cakupan tes, dan pembaruan dokumentasi.

3.1.6 Cognition Devin

Devin berada di ujung spektrum otonomi penuh. Agen ini menyusun rencana, menulis kode lintas berkas, menyiapkan lingkungan, menjalankan tes, memperbaiki *bug* miliknya sendiri, lalu membuka *pull request* tanpa intervensi manusia di tengah proses. Hal yang menarik bagi mahasiswa adalah harga masuknya turun drastis, dari kelas 500 dolar AS per bulan ke paket inti 20 dolar AS per bulan dengan komputasi bayar sesuai pemakaian. Dengan demikian, eksperimen pembanding menjadi terjangkau.

3.1.7 Sistem Akademik: MetaGPT, ChatDev, dan SWE-agent

- MetaGPT menyandikan prosedur operasi baku (SOP) tim perangkat lunak ke dalam urutan *prompt* dengan peran *Product Manager*, *Architect*, dan *Engineer*. Bersama GPT-4, sistem ini mencapai Pass@1 sebesar 85,9 persen pada HumanEval dan 87,7 persen pada MBPP, serta tingkat penyelesaian tugas 100 persen pada dataset SoftwareDev milik mereka.
- ChatDev menempatkan agen berperan CEO, CTO, *Programmer*, *Reviewer*, dan *Tester* yang berkolaborasi melalui rantai percakapan (*chat chain*) dengan meniru model *waterfall*.
- SWE-agent adalah agen tunggal berbasis LLM dengan *Agent-Computer Interface* (ACI) untuk membaca, menyunting, dan mengeksekusi kode guna menyelesaikan isu GitHub nyata pada *benchmark* SWE-bench.

Sebagai kalibrasi kemampuan mutakhir, pada 2026 skor SWE-bench Verified model *frontier* telah berada di kisaran 90-an persen sehingga papan peringkat mendekati titik jenuh. Auditor *benchmark* memberikan catatan penting bahwa skor yang tinggi harus dibaca dengan waspada terhadap kontaminasi data

latih dan kualitas kasus uji. Implikasi desain bagi proyek ini adalah kemampuan menghasilkan *patch* tidak lagi menjadi hambatan utama. Hambatan bergeser ke orkestrasi, tata kelola, dan verifikasi.

3.2 Analisis Perbandingan

Perbandingan antara produk yang ada dan T2D-MAS dibagi menjadi dua tabel agar terbaca dengan jelas. Tabel 8 membandingkan cakupan alur, taksonomi agen, agen bergerak, dan tata kelola. Tabel 9 membandingkan integrasi RL dan GNN, ketergantungan vendor, dan kematangan produksi.

Tabel 8. Perbandingan cakupan, taksonomi, agen bergerak, dan tata kelola

Kriteria	Atlassian Rovo	ServiceNow AI Agents	watsonx Orchestrate	Copilot Agent / Devin	MetaGPT / ChatDev	T2D-MAS (usulan)
Cakupan alur	Tiket sampai tinjauan PR	Insiden ITSM sampai remediasi	Proses bisnis lintas aplikasi	Isu sampai PR	Kebutuhan sampai kode	Tiket, *deploy*, pemantauan, dan umpan balik (*loop* tertutup)
Taksonomi agen eksplisit	Sebagian (agen bawaan)	Ya (*studio*)	Ya (katalog)	Tidak (agen tunggal)	Ya (peran SOP)	Ya: *Scout*, *Broker*, *Worker*, *Security*
Agen bergerak (kode berpindah ke data)	Tidak	Tidak	Tidak	Efemeral, tetapi di infrastruktur vendor	Tidak	Ya, eksplisit dan terukur
Granularitas tata kelola	Tingkat otomasi	AI Control Tower (organisasi)	Tata kelola terpusat	Aturan *branch* dan PR	Minim	Per peran dan aturan gerbang (τ, R_{maks})

Tabel 9. Perbandingan integrasi RL, GNN, ketergantungan vendor, dan kematangan

Kriteria	Atlassian Rovo	ServiceNow AI Agents	watsonx Orchestrate	Copilot Agent / Devin	MetaGPT / ChatDev	T2D-MAS (usulan)
Integrasi RL	Tidak dipublikasikan	Tidak dipublikasikan	Tidak dipublikasikan	Tidak	Tidak	Ya (prioritisasi tes dan kebijakan rilis)
Integrasi GNN	Tidak	Tidak	Tidak	Tidak	Tidak	Ya (lokalisasi akar masalah)

Kriteria	Atlassian Rovo	ServiceNow AI Agents	watsonx Orchestrate	Copilot Agent / Devin	MetaGPT / ChatDev	T2D-MAS (usulan)
Ketergantungan vendor	Tinggi (ekosistem Atlassian)	Tinggi	Sedang (protokol terbuka)	Tinggi	Rendah (riset)	Rendah (sumber terbuka)
Kematangan produksi	Tinggi	Tinggi	Tinggi	Tinggi	Rendah sampai sedang	Purwarupa akademik

3.3 Celah yang Diisi Proyek Ini

Berdasarkan perbandingan tersebut, proyek ini mengisi empat celah berikut.

1. **Alur belum tertutup.** Produk komersial unggul pada satu segmen, yaitu Rovo pada triase dan PR, ServiceNow pada insiden, serta Copilot dan Devin pada pengodean. Namun, belum ada satu sistem yang secara publik memodelkan rantai penuh umpan balik, triase, investigasi, perencanaan, implementasi, verifikasi, *deployment*, pemantauan, lalu kembali ke triase dengan satu keadaan bersama (*shared state*).
2. **Semua agen pada dasarnya statis.** Rovo, ServiceNow, dan Copilot menjalankan komputasi di lingkungan vendor dan menarik data ke sana. Tidak ada mekanisme resmi untuk memindahkan agen ke dalam simpul produksi pelanggan agar pemrosesan log terjadi secara lokal. Padahal, keunggulan klasik paradigma agen bergerak justru terletak di sini, yaitu memindahkan komputasi ke data, bukan sebaliknya. Dengan cara itu, beban jaringan berkurang, latensi teratasi, dan eksekusi asinkron pada koneksi yang tidak andal menjadi mungkin.
3. **Tata kelola masih kasar.** Persetujuan umumnya bersifat biner per alur, bukan fungsi dari keyakinan model dan estimasi risiko per kelas keputusan.
4. **Ragam metode AI masih terbatas.** Pemanfaatan RL untuk prioritisasi pengujian dan GNN untuk lokalisasi akar masalah sudah matang dalam literatur, tetapi belum terlihat menyatu ke dalam produk agen komersial.

3.4 Kelebihan dan Kekurangan Konsep yang Diusulkan

Konsep yang kami usulkan memiliki empat kelebihan. Pertama, konsep ini menerapkan *loop* tertutup dengan satu *blackboard*. Kedua, taksonomi agennya eksplisit sehingga otonomi dapat diatur per peran. Ketiga, mobilitas agen memberikan keuntungan terukur pada biaya komunikasi. Keempat, portofolio metodenya hibrida (berbasis aturan, ML, DL, RL/GNN, hingga LLM) dan dipilih pada setiap tahap berdasarkan pertimbangan untung-rugi, bukan seragam memakai LLM.

Konsep ini juga memiliki kekurangan. Konsep ini belum teruji pada skala produksi. Mobilitas agen menambah permukaan serangan yang harus dijawab dengan *sandboxing* dan penandatanganan kode. Konsep ini bergantung pada kualitas data historis perusahaan. Biaya rekayasa awalnya jauh lebih tinggi daripada sekadar berlangganan Rovo atau ServiceNow. Untuk organisasi kecil dengan satu jenis tugas, misalnya hanya perbaikan *bug* dari satu isu GitHub, pendekatan agen tunggal seperti SWE-agent tetap lebih rasional karena beban koordinasi antaragen lebih rendah.

BAB 4

TUJUAN PROYEK

4.1 Tujuan Umum

Tujuan umum proyek ini adalah merancang dan memprototipekan sebuah sistem multiagen cerdas tingkat *enterprise* yang mengotomasi sebagian besar tahap alur *Ticket-to-Deployment*. Sistem ini tetap mempertahankan titik pemeriksaan manusia pada keputusan berisiko tinggi dan meningkatkan otonomi secara bertahap mengikuti kerangka *Capability Expansion* (Era 1 sampai 8).

4.2 Tujuan Khusus

Proyek ini memiliki enam tujuan khusus, yaitu T1 sampai T6. Setiap tujuan disertai bukti pemenuhan dan bab yang membahasnya, seperti tersaji pada Tabel 10.

Tabel 10. Tujuan khusus proyek

Kode	Tujuan	Bukti pemenuhan	Bab
T1	Mahasiswa mampu mendesain konsep sistem multiagen cerdas untuk *enterprise*	Taksonomi agen, diagram arsitektur, dan diagram koordinasi	5
T2	Mengintegrasikan ML, DL, dan RL ke dalam agen	Modul klasifikasi (ML), deduplikasi dan *severity* (DL), prioritisasi tes dan kebijakan rilis (RL)	6
T3	Menyediakan UI/UX yang jelas	Dasbor pemantauan agen, status, dan jejak agen (*agent trace*)	8
T4	Memasukkan agen cerdas bergerak dan membuktikan keunggulannya atas agen statis	Klasifikasi mobilitas, perhitungan biaya komunikasi, dan eksperimen	5.4, 9
T5	Menyediakan tata kelola otonomi yang dapat diaudit	Aturan gerbang (τ, R_{maks}), tabel *Autonomy Boundary*, dan *audit log*	5.7, 6.8
T6	Mengevaluasi sistem terhadap *baseline* statis secara kuantitatif	Tabel hasil eksperimen empat konfigurasi	9

4.3 Target Terukur (SMART)

Target terukur prototipe disajikan pada Tabel 11 beserta nilai *baseline* dan cara pengukurannya.

Tabel 11. Target terukur prototipe

Metrik	*Baseline* (manual/statis)	Target prototipe	Cara ukur
Waktu triase per tiket	12 menit	≤ 30 detik	Stempel waktu masuk sampai berstatus *triaged*
Recall@10 deteksi duplikat	sekitar 0,45 (kata kunci)	≥ 0,80	*Dataset* berlabel (Subbab 7.3)
Macro-F1 klasifikasi *severity*	-	≥ 0,75	Empat kelas, *test set* terstratifikasi
Byte log berpindah melalui jaringan per investigasi	8.192 MB	≤ 100 MB	Instrumentasi *network counter*
MTTR (simulasi)	100 persen (relatif)	≤ 60 persen	Rata-rata waktu tiket sampai *deploy*
Pelanggaran kebijakan yang lolos ke produksi	-	0	Uji injeksi kebijakan (Subbab 9.2)
Keputusan otonom yang dibatalkan manusia	-	≤ 10 persen	Log *override* pada dasbor

4.4 Ruang Lingkup dan Batasan

Termasuk dalam lingkup proyek adalah simulasi alur sembilan tahap, empat kelas agen dengan sembilan instansiasi, integrasi model ML/DL/RL/GNN pada skala *dataset* publik, dasbor pemantauan, simulasi migrasi agen antarsimpul (kontainer lokal), dan *audit log* yang lengkap.

Tidak termasuk dalam lingkup proyek adalah *deployment* ke infrastruktur produksi yang sebenarnya, *fine-tuning* LLM besar dari nol, integrasi ke sistem berbayar (Jira Cloud Premium dan Datadog) yang diganti dengan API tiruan (*mock*) dan *dataset* publik, serta jaminan keamanan tingkat produksi untuk mobilitas kode.

Asumsi yang dipakai adalah bahwa *dataset* publik laporan *bug* cukup representatif terhadap tiket perusahaan *e-commerce* setelah skema dinormalisasi. Biaya inferensi LLM diabaikan pada skala purwarupa, tetapi dihitung sebagai metrik pada Bab 9.

BAB 5

DESAIN SISTEM MULTIAGEN

5.1 Spesifikasi Sistem dan Fitur

5.1.1 Kebutuhan Fungsional

Sistem T2D-MAS memiliki lima belas kebutuhan fungsional, dari F-01 sampai F-15. Setiap kebutuhan dilaksanakan oleh agen tertentu, seperti tersaji pada Tabel 12.

Tabel 12. Kebutuhan fungsional

Kode	Fitur	Deskripsi	Agen pelaksana
F-01	*Feedback capture* multikanal	Menerima tiket dari portal, surel, Slack, dan *crash reporter*	*Scout-Feedback*
F-02	Normalisasi dan pengayaan tiket	Mengekstraksi entitas, versi aplikasi, perangkat, dan langkah reproduksi	*Scout-Feedback*
F-03	Deteksi duplikat semantik	Mencocokkan vektor terhadap riwayat tiket dengan ambang τ_{dup}	*Broker-Triage*
F-04	Klasifikasi kategori dan *severity*	Empat kelas *severity* dan kategori komponen	*Broker-Triage*
F-05	Pengarahan dan alokasi skuad	Melelang tugas (*contract net*) kepada *Worker* berdasarkan keahlian dan beban	*Broker-Assign*
F-06	Investigasi otomatis di lokasi data	Menganalisis log dan metrik pada simpul produksi serta mengorelasikan jejak	*Scout-Log* (mobile)
F-07	Lokalisasi akar masalah	Memeringkat layanan tersangka pada graf dependensi	*Worker-Investigate*
F-08	Dekomposisi tugas dan estimasi	Menyusun subtugas, kriteria penerimaan, dan estimasi usaha	*Worker-Plan*
F-09	Generasi kode dan draf PR	Menghasilkan *patch*, tes unit, dan deskripsi PR	*Worker-Impl* (mobile)
F-10	Prioritisasi dan eksekusi pengujian	Memilih subset tes bernilai tertinggi dan menganalisis regresi	*Worker-QA*
F-11	Eksekusi rilis dan *rollback*	Menjalankan *canary* atau *full deploy*, memantau ambang, dan melakukan *rollback* otomatis	*Worker-Deploy*
F-12	Telemetri dan deteksi anomali	Memantau SLO/KPI dan anomali produksi serta membuat tiket umpan balik	*Scout-Monitor* (mobile)
F-13	Penegakan kebijakan dan audit	Memindai rahasia, menjalankan SAST, mengevaluasi aturan gerbang, dan mencatat jejak audit	*Security-Policy*
F-14	Dasbor pemantauan agen	Menampilkan status agen, antrean, jejak keputusan, dan tombol *override*	UI

Kode	Fitur	Deskripsi	Agen pelaksana
F-15	Laporan kinerja	Menyajikan metrik DORA dan metrik agen serta ekspor CSV	UI dan *Scout-Monitor*

5.1.2 Kebutuhan Nonfungsional

Selain kebutuhan fungsional, sistem memiliki tujuh kebutuhan nonfungsional, dari NF-01 sampai NF-07, yang tersaji pada Tabel 13.

Tabel 13. Kebutuhan nonfungsional

Kode	Aspek	Target
NF-01	Latensi triase	p95 ≤ 30 detik sejak tiket masuk
NF-02	Skalabilitas	Sekurang-kurangnya 50 tiket diproses secara paralel pada satu simpul 8 vCPU
NF-03	Keamanan	Semua agen bergerak berjalan dalam *sandbox*, kode ditandatangani, dan kredensial berumur pendek
NF-04	Auditabilitas	Setiap aksi otonom tercatat, meliputi aktor, masukan, keyakinan, dan kebijakan yang berlaku
NF-05	Observabilitas	Setiap pesan antaragen memiliki *correlation id*
NF-06	Degradasi anggun	Kegagalan satu agen tidak menghentikan alur, dan tiket jatuh ke antrean manusia
NF-07	Privasi	Log yang keluar dari simpul produksi hanya berupa fitur teragregasi, dengan PII yang disamarkan

5.2 Justifikasi Agen Tunggal dan Multiagen

Alur ini terdiri atas sembilan tahap yang menuntut keahlian dan *tool* yang sangat berbeda. Triase membutuhkan NLP, investigasi membutuhkan akses log dan observabilitas, implementasi membutuhkan generasi kode, verifikasi membutuhkan *test runner*, dan rilis membutuhkan API *deployment*. Kami memilih arsitektur multiagen dengan empat alasan.

1. **Spesialisasi mengurangi beban konteks.** Setiap agen hanya memuat konteks dan *tool* yang relevan dengan tahapnya sehingga risiko instruksi tercampur berkurang dibandingkan dengan satu agen yang menangani seluruh alur.
2. **Mengurangi halusinasi berantai.** Riset MetaGPT menunjukkan bahwa perantaian LLM secara naif menyebabkan halusinasi menumpuk antarlangkah, dan pembagian peran dengan SOP yang jelas menekan masalah ini.

3. *Paralelisasi dan* throughput*. Beberapa tiket dapat diproses secara bersamaan oleh instans agen yang berbeda*
4. **Tata kelola yang granular.** Setiap peran dapat memiliki batas otonomi yang berbeda. Sebagai contoh, *Worker-Investigate* boleh otonom penuh, sedangkan *Worker-Deploy* wajib menunggu persetujuan untuk rilis berdampak besar.

Untuk lingkup yang lebih kecil atau tugas yang berdiri sendiri, pendekatan agen tunggal seperti SWE-agent tetap relevan karena beban koordinasinya lebih rendah. Proyek ini memilih multiagen karena cakupannya adalah seluruh siklus, bukan satu tugas pengodean tunggal.

5.3 Taksonomi Agen: Scout, Broker, Worker, dan Security

Tata letak tugas mensyaratkan empat arketipe agen. Tabel 14 memetakan keempat arketipe itu ke alur *Ticket-to-Deployment* sehingga rancangan lama tetap terpakai sekaligus memenuhi struktur yang diminta.

Tabel 14. Taksonomi agen dan instansiasinya

Arketipe	Peran generik	Instansiasi pada T2D-MAS	Tugas konkret
Scout Agent	Pengumpulan data dan pengindraan lingkungan	*Scout-Feedback*, *Scout-Log*, *Scout-Monitor*	Menyerap tiket multikanal; menambang log dan metrik di simpul produksi; memantau SLO dan mendeteksi anomali
Broker Agent	Pencocokan dan alokasi sumber daya	*Broker-Triage*, *Broker-Assign*	Mencocokkan tiket dengan riwayat (deduplikasi), menetapkan kategori dan *severity*, serta melelang tugas kepada *Worker* sesuai keahlian dan beban
Worker Agent	Eksekusi tugas dan pelaporan	*Worker-Investigate*, *Worker-Plan*, *Worker-Impl*, *Worker-QA*, *Worker-Deploy*	Menganalisis akar masalah, mendekomposisi tugas, menghasilkan *patch*, menjalankan pengujian, serta merilis dan melakukan *rollback*
Security Agent	Kebijakan dan kepatuhan	*Security-Policy*	Memindai rahasia dan kerentanan, mengevaluasi aturan gerbang otonomi, menandatangani dan memverifikasi agen bergerak, serta menulis jejak audit

5.3.1 Komponen Internal Setiap Agen

Setiap agen dibangun dari lima komponen kanonik arsitektur agen cerdas, yaitu (1) *perception/input*, (2) *memory/state*, (3) *reasoning/planning*, (4) *tool-use/action interface*, dan (5) *output*. Tabel 15 menguraikan tiga komponen pertama dan Tabel 16 menguraikan dua komponen berikutnya untuk setiap agen.

Tabel 15. Komponen internal agen: masukan, memori, dan inti penalaran

Agen	Masukan / persepsi	Memori / keadaan	Inti penalaran
Scout-Feedback	Teks atau gambar tiket mentah, surel, dan *webhook*	*Buffer* kanal dan deduplikasi ID pesan	NER dan normalisasi skema
Broker-Triage	Tiket ternormalisasi	Riwayat tiket serupa (*vector DB*)	Pengklasifikasi ML/DL dan skor kemiripan (Subbab 6.3 dan 6.4)
Broker-Assign	Tiket tertriase dan status skuad	Beban dan matriks keahlian *Worker*	*Contract Net Protocol* (Subbab 5.6)
Scout-Log (mobile)	Log dan metrik mentah di simpul	*Baseline* lokal dan *checkpoint* posisi log	Filter, agregasi, dan deteksi pola
Worker-Investigate	Tiket, bukti awal, dan fitur dari *Scout-Log*	Log historis dan basis pengetahuan insiden	GNN (lokalisasi) dan penalaran LLM berantai
Worker-Plan	Laporan investigasi	Status *backlog* dan *sprint*	*Planner* dekomposisi tugas
Worker-Impl (mobile)	Spesifikasi tugas	Konteks kode (RAG atas repositori)	Generasi kode LLM dan perbaikan mandiri (*self-repair*)
Worker-QA	PR dan spesifikasi tes	Riwayat hasil tes dan daftar tes tidak stabil (*flaky*)	Prioritisasi tes dengan RL dan analisis hasil
Worker-Deploy	PR yang telah disetujui	Riwayat *deployment* dan *rollback*	Penilai risiko rilis dan kebijakan *canary* RL
Scout-Monitor (mobile)	Telemetri produksi	*Baseline* SLO/SLA	Deteksi anomali (statistik dan DL)
Security-Policy	Seluruh permintaan aksi berisiko	Katalog kebijakan dan riwayat pelanggaran	*Rule engine* dan aturan gerbang (Subbab 6.8)

Tabel 16. Komponen internal agen: *tool* dan keluaran

Agen	*Tool* / antarmuka aksi	Keluaran
Scout-Feedback	API surel dan Slack, OCR	Tiket ternormalisasi
Broker-Triage	API Jira/GitHub dan pencarian vektor	Kategori, *severity*, tim tujuan, dan penanda duplikat
Broker-Assign	API manajemen proyek	Penetapan *Worker* dan SLA
Scout-Log (mobile)	Pembaca log lokal, eBPF/*tail*, penyamar PII	Vektor fitur terkompresi dan kutipan bukti
Worker-Investigate	API observabilitas dan graf dependensi	Peringkat akar masalah dan langkah reproduksi

Agen	*Tool* / antarmuka aksi	Keluaran
Worker-Plan	API Jira/manajemen proyek	Subtugas, kriteria penerimaan, dan estimasi usaha
Worker-Impl (mobile)	Git, operasi berkas, dan *test runner* (*sandbox* efemeral)	*Code diff* atau draf PR
Worker-QA	API CI/CD dan *test framework*	Laporan lulus/gagal dan penanda regresi
Worker-Deploy	Alur *deployment* CI/CD dan API infrastruktur	Eksekusi *deployment* dan pemicu *rollback*
Scout-Monitor (mobile)	*Stack* observabilitas	Peringatan dan tiket umpan balik baru
Security-Policy	Pemindai rahasia, SAST, penandatangan, dan penyimpanan audit	Izin, penolakan, atau eskalasi beserta entri audit

5.4 Klasifikasi Agen Bergerak dan Agen Statis

5.4.1 Definisi Operasional

- **Agen statis** berjalan secara permanen pada *host* orkestrator dan mengakses sumber daya jarak jauh melalui RPC atau API. Pada agen statis, data berpindah ke komputasi.
- *Agen bergerak (*mobile agent*) mengemas kode dan keadaannya, berpindah ke* host *tujuan (simpul produksi,* CI runner*, atau* edge*), melanjutkan eksekusi di sana, lalu mengirim kembali hasil yang ringkas. Pada agen bergerak, komputasi berpindah ke data.
- **Agen AI** adalah agen, baik statis maupun bergerak, yang inti penalarannya berupa model pembelajaran (ML, DL, RL, atau LLM), bukan aturan tetap.

5.4.2 Tabel Klasifikasi

Klasifikasi mobilitas setiap agen, inti penalarannya, dan alasan pemilihannya tersaji pada Tabel 17.

Tabel 17. Klasifikasi mobilitas agen

Agen	Mobilitas	Inti penalaran	Alasan pemilihan mobilitas
Scout-Feedback	Statis	ML ringan dan aturan	Sumber data sudah terpusat di kanal
Broker-Triage	Statis	DL (SBERT dan pengklasifikasi)	Membutuhkan pandangan global atas seluruh riwayat tiket
Broker-Assign	Statis	Optimasi dan aturan	Membutuhkan keadaan global beban seluruh *Worker*

Agen	Mobilitas	Inti penalaran	Alasan pemilihan mobilitas
Scout-Log	Bergerak	Statistik dan DL ringan	Log mentah bervolume gigabita berada di simpul produksi, dan PII tidak boleh keluar
Worker-Investigate	Statis	GNN dan LLM	Memerlukan graf dependensi lintas layanan (global)
Worker-Plan	Statis	LLM	Bekerja pada artefak teks, bukan data besar
Worker-Impl	Bergerak	LLM dan *self-repair*	Berpindah ke *sandbox* efemeral berisi repositori dan *toolchain*; pola yang sama dipakai oleh Copilot *coding agent* dan Devin
Worker-QA	Statis (koordinator) dan bergerak (eksekutor)	RL	Eksekusi tes dikirim ke *runner* terdekat dengan artefak *build*
Worker-Deploy	Statis	RL dan aturan	Aksi berisiko tinggi harus terjadi pada jalur yang terkendali dan teraudit
Scout-Monitor	Bergerak	Deteksi anomali	Telemetri berumur pendek dan bervolume besar; agregasi di tepi jauh lebih murah
Security-Policy	Statis dan pengawal (*escort*)	*Rule engine*	Kebijakan harus konsisten; namun paket kebijakan yang ditandatangani ikut berpindah bersama agen bergerak

5.4.3 Alasan Agen Bergerak Lebih Baik daripada Agen Statis: Perhitungan Biaya Komunikasi

Literatur klasik paradigma agen bergerak menyebutkan bahwa alasan utamanya adalah mengurangi beban jaringan dengan memindahkan kode mendekati data alih-alih memindahkan data dalam jumlah besar ke klien. Paradigma ini sekaligus mengatasi latensi jaringan dan memungkinkan eksekusi asinkron pada koneksi yang tidak andal. Berikut ini adalah kuantifikasi untuk satu siklus investigasi insiden yang didasarkan pada asumsi.

Parameter yang digunakan adalah jumlah simpul produksi yang perlu diperiksa n = 6, volume log per simpul dalam jendela insiden L = 1.365 MB (total sekitar 8.192 MB atau 8 GB), ukuran paket agen bergerak (kode, model ringan, dan keadaan) A = 12 MB, ukuran hasil ringkas per simpul (vektor fitur dan kutipan bukti) R = 2 MB, serta lebar pita efektif tautan B = 100 Mbps atau 12,5 MB/detik.

Skenario A: agen statis (menarik log ke pusat). Data yang berpindah adalah hasil kali jumlah simpul dan volume log per simpul, dan waktu transfer adalah data yang berpindah dibagi lebar pita.

$$D_{statis} = n \times L = 6 \times 1.365\,MB = 8.190\,MB \quad (6)$$

$$T_{statis} = \frac{8.190}{12{,}5} \approx 655\,detik \approx 10,9\,menit \quad (7)$$

Skenario B: agen bergerak (mengirim agen ke simpul). Data yang berpindah adalah hasil kali jumlah simpul dan jumlah ukuran agen serta ukuran hasil. Total waktu terdiri atas waktu transfer dan waktu komputasi lokal, yang berjalan paralel pada enam simpul dan diasumsikan sekitar 20 detik.

$$D_{mobile} = n \times (A + R) = 6 \times (12 + 2) = 84\,MB \quad (8)$$

$$T_{mobile} = \frac{84}{12{,}5} + 20 \approx 6,7 + 20 \approx 27\,detik \quad (9)$$

Dari kedua skenario tersebut, penghematan data dan percepatan waktu dihitung sebagai berikut.

$$Reduksi\ byte = \frac{8.190 - 84}{8.190} \approx 98,97\% \quad (10)$$

$$Speedup\ waktu = \frac{655}{27} \approx 24\times \quad (11)$$

Ringkasan perbandingan tersaji pada Tabel 18.

Tabel 18. Perbandingan biaya komunikasi agen statis dan agen bergerak (asumsi)

Uraian	Agen statis	Agen bergerak
Data berpindah	8.190 MB	84 MB
Waktu transfer	sekitar 655 detik (10,9 menit)	sekitar 6,7 detik
Komputasi lokal (paralel enam simpul)	-	sekitar 20 detik (asumsi)
Total waktu	sekitar 655 detik	sekitar 27 detik

Titik impas (*break-even*) terjadi ketika biaya migrasi sama dengan biaya penarikan data, yaitu ketika n(A + R) = nL. Titik ini tercapai saat volume log per simpul turun sampai sekitar 14 MB. Di bawah nilai itu, agen statis lebih efisien karena biaya migrasi tidak terbayar. Kesimpulan desainnya adalah bahwa mobilitas bukan dogma, melainkan keputusan yang bergantung pada rasio volume

data terhadap ukuran agen. Aturan yang dipakai T2D-MAS adalah sebagai berikut.

$$L_{node} > (A + R) \times k \quad (12)$$

Pada aturan tersebut, k adalah faktor keamanan dengan nilai 3. Untuk A + R = 14 MB, ambang migrasinya adalah 42 MB per simpul. Selain keuntungan kuantitatif, agen bergerak memberikan dua keuntungan nonkuantitatif, yaitu ketahanan terhadap putusnya koneksi (agen tetap bekerja lalu melapor saat tautan pulih) dan kepatuhan privasi (data mentah tidak pernah meninggalkan batas produksi).

5.4.4 Risiko Mobilitas dan Mitigasinya

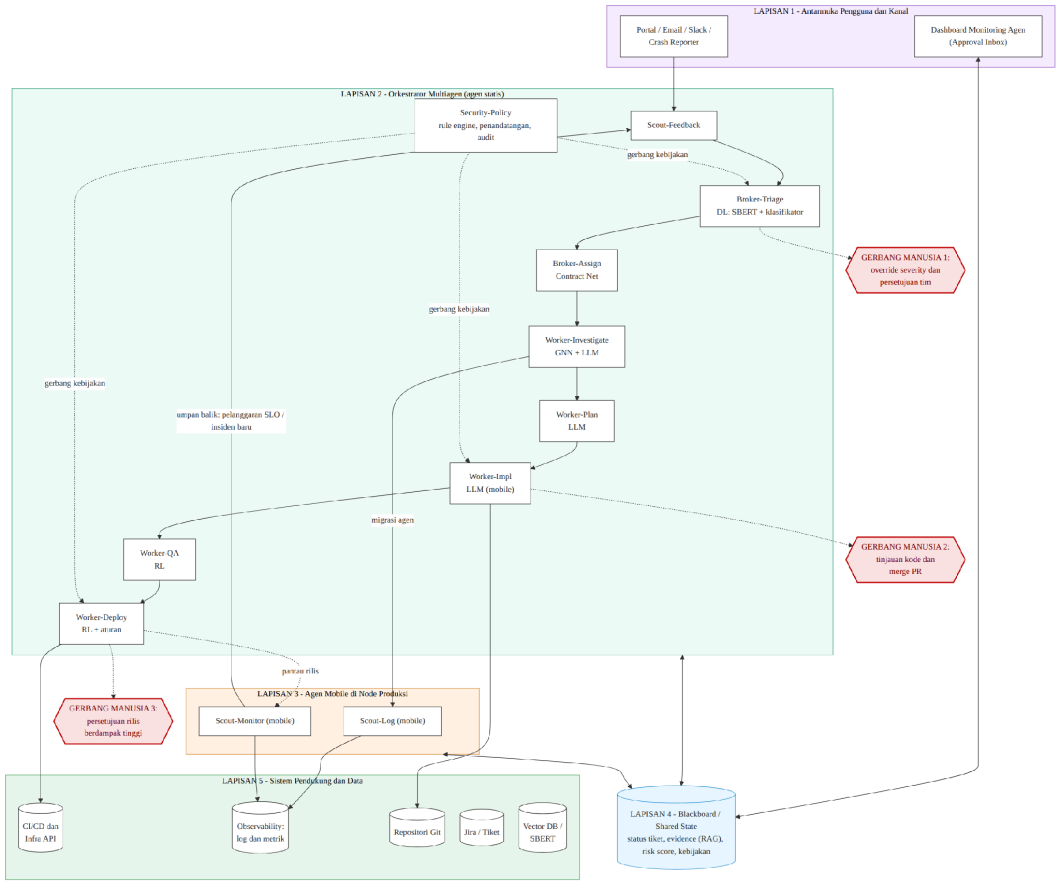
Mobilitas kode menimbulkan risiko baru yang harus dimitigasi. Risiko dan mitigasinya tersaji pada Tabel 19.

Tabel 19. Risiko mobilitas agen dan mitigasinya

Risiko	Mitigasi
Kode berbahaya berpindah ke simpul produksi	*Security-Policy* menandatangani kode dan memverifikasi tanda tangan sebelum eksekusi
Agen memakai sumber daya secara berlebihan di simpul	Kuota cgroup (CPU, RAM, I/O) dan *watchdog* dengan TTL
Agen dimodifikasi pada *host* yang tidak tepercaya	Keadaan di-*hash* dan ditandatangani ulang setiap perpindahan, serta hasil diverifikasi silang
Kebocoran data saat agen kembali	*Egress filter*: hanya skema hasil yang telah didefinisikan (*allowlist* bidang) yang boleh dikirim
Agen "tersesat" atau yatim	Registri migrasi dan *heartbeat*; agen tanpa *heartbeat* lebih dari 60 detik dimusnahkan

5.5 Diagram Arsitektur Sistem

Gambar 1 menampilkan arsitektur sistem T2D-MAS yang terdiri atas lima lapisan. Lapisan 1 adalah antarmuka pengguna dan kanal. Lapisan 2 adalah orkestrator multiagen yang berisi agen statis. Lapisan 3 adalah agen bergerak di simpul produksi. Lapisan 4 adalah keadaan bersama atau *blackboard*. Lapisan 5 adalah sistem pendukung dan data.



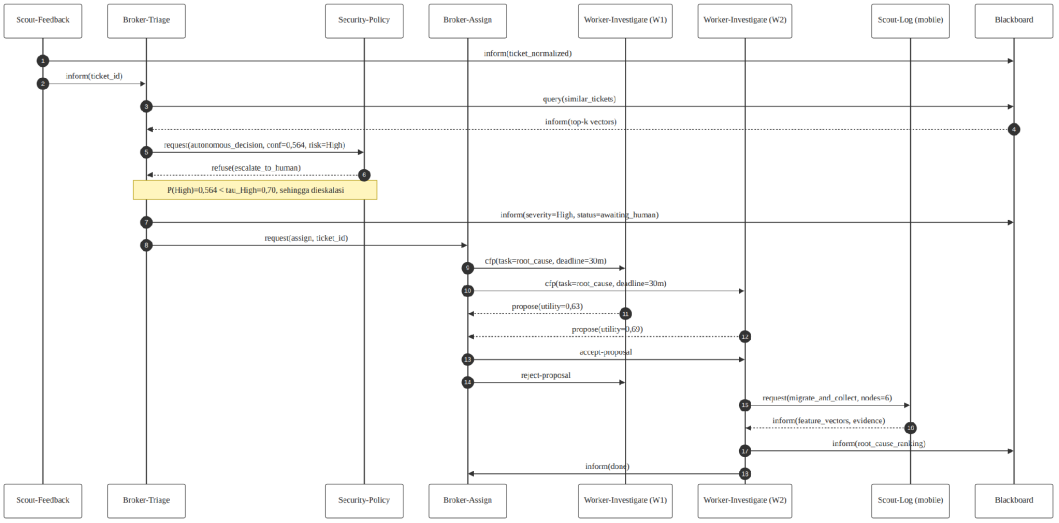
Gambar 1. Arsitektur sistem T2D-MAS

Tiga gerbang persetujuan manusia dipertahankan dari desain sebelumnya dan ditandai merah pada diagram. Ketiga gerbang itu adalah sebagai berikut.

1. Penggantian *severity* (*override*) dan persetujuan tim setelah *Broker-Triage*.
2. Tinjauan kode produksi dan penggabungan PR setelah *Worker-Impl*.
3. Persetujuan rilis berdampak tinggi sebelum *Worker-Deploy* mengeksekusi.

5.6 Diagram Koordinasi, Kolaborasi, dan Negosiasi (MIMA)

Koordinasi antaragen memakai pengiriman pesan bergaya FIPA-ACL dengan tujuh performatif, yaitu *cfp* (*call for proposal*), *propose*, *refuse*, *accept-proposal*, *reject-proposal*, *inform*, dan *failure*. Pola alokasi tugas memakai *Contract Net Protocol* (CNP). Gambar 2 menunjukkan urutan pesan pada satu tiket, mulai dari penerimaan tiket, eskalasi ke manusia, pelelangan tugas, sampai migrasi agen bergerak untuk pengumpulan bukti.



Gambar 2. Diagram koordinasi dan negosiasi (MIMA)

5.6.1 Contoh Numerik Negosiasi (Contract Net)

Broker-Assign melelang tiket TCK-1042 yang berkategori *severity* High dengan komponen *backend*. Fungsi utilitas tawaran adalah sebagai berikut, dengan bobot $w_1 = 0{,}5$, $w_2 = 0{,}3$, dan $w_3 = 0{,}2$.

$$U = w_1 \cdot skill_{match} + w_2 \cdot (1 - load) + w_3 \cdot (1 - cost_{norm}) \quad (13)$$

Perhitungan utilitas ketiga *Worker* tersaji pada Tabel 20.

Tabel 20. Perhitungan utilitas tawaran pada tiket TCK-1042

Worker	skill_match	load	cost_norm	Perhitungan dan U
W1	0,90	0,80	0,40	0,45 + 0,06 + 0,12 = 0,63
W2	0,70	0,20	0,50	0,35 + 0,24 + 0,10 = 0,69
W3	0,50	0,10	0,30	0,25 + 0,27 + 0,14 = 0,66

Pemenangnya adalah W2. Contoh ini penting secara konseptual karena agen yang paling ahli (W1) tidak selalu menang. Beban kerja W1 yang tinggi membuat SLA berisiko. Inilah perbedaan mendasar antara pengarahan berbasis aturan ("selalu kirim ke tim *backend*") dan negosiasi berbasis utilitas.

5.6.2 Skema Pesan

Setiap pesan antaragen mengikuti skema JSON yang memuat performatif, pengirim, penerima, isi, konteks kebijakan, dan tanda tangan digital. Contoh pesan *cfp* untuk tiket TCK-1042 adalah sebagai berikut.

```
{
  "message_id": "msg_01H9Z...",
  "correlation_id": "TCK-1042",
  "performative": "cfp",
  "sender": "broker-assign@orchestrator",
  "receiver": ["worker-investigate-1", "worker-investigate-2"],
  "ontology": "t2d-mas/v1",
  "content": {
    "task": "root_cause_analysis",
    "ticket_id": "TCK-1042",
    "severity": "High",
    "deadline_s": 1800,
    "required_skills": ["backend", "payment", "postgres"]
  },
  "policy_context": {"tau": 0.70, "risk_max": 0.60},
  "signature": "ed25519:..."
}
```

5.7 Batas Otonomi dan Tata Kelola (*Autonomy Boundary and Governance*)

Tabel 21 menetapkan kapan setiap keputusan boleh diambil secara otonom, kapan persetujuan manusia wajib, dan apakah keputusan tersebut dapat dibatalkan (*rollback*).

Tabel 21. Batas otonomi dan tata kelola

Keputusan	Agen	Otonom jika	Persetujuan manusia wajib bila	Dapat di-*rollback*?
Menandai duplikat	*Broker-Triage*	cos ≥ 0,85	0,70 ≤ cos < 0,85	Ya (*unmerge*)
Menetapkan *severity*	*Broker-Triage*	P(kelas) ≥ 0,70 dan kelas bukan Critical	Kelas Critical, atau P < 0,70	Ya
Menetapkan *Worker*	*Broker-Assign*	Selalu	-	Ya
Menarik log produksi	*Scout-Log*	Selalu (hasil teragregasi)	Jika perlu akses bidang PII	Tidak berlaku
Membuat draf PR	*Worker-Impl*	Selalu (draf, bukan *merge*)	-	Ya
Merge ke *main*	*Worker-Impl*	Tidak pernah	Selalu (tinjauan kode)	Ya
Menjalankan tes	*Worker-QA*	Selalu	-	Tidak berlaku
Deploy canary ≤ 5 persen	*Worker-Deploy*	risk_score ≤ 0,3	risk_score > 0,3	Ya (*rollback* otomatis)
Deploy penuh ke produksi	*Worker-Deploy*	Tidak pernah	Selalu (persetujuan rilis)	Ya
Rollback otomatis	*Worker-Deploy*	Selalu (*fail-safe*)	-	-
Mengubah aturan kebijakan	*Security-Policy*	Tidak pernah	Selalu (dua orang, prinsip empat mata)	Ya

BAB 6

ALGORITMA DAN INTEGRASI AI

6.1 Prinsip Pemilihan Metode

Pemilihan metode mengikuti kerangka *Capability Expansion* (Era 1 sampai 8). Metode dipilih berdasarkan pertimbangan akurasi, keterjelasan (*explainability*), dan biaya komputasi, bukan otomatis memakai LLM di semua tempat. Hasilnya adalah pendekatan hibrida dan bertahap, bukan lompatan langsung ke Era 8. Pemetaan metode pada setiap tahap alur tersaji pada Tabel 22.

Tabel 22. Pemetaan tahap alur ke metode

Tahap alur	Metode	Era	Justifikasi singkat
Validasi bidang dan pewaktu SLA	Berbasis aturan	1	Membutuhkan determinisme dan jejak audit yang mudah dijelaskan kepada auditor
Prediksi dampak/*churn* dan pengarahan kasar	ML klasik (RF/SVM)	2	Fitur tabular terstruktur, cepat dilatih, dan mudah diinterpretasi
Deteksi duplikat dan klasifikasi *severity*	DL (SBERT, CNN, BERT)	3	Menangkap kemiripan semantik yang lolos dari pencocokan kata kunci
Ringkasan, kriteria penerimaan, catatan rilis, dan kode	*Generative AI* / LLM	4	Perlu menghasilkan bahasa alami atau kode
Prioritisasi tes dan kebijakan rilis	RL	5	Keputusan berurutan dengan umpan balik tertunda
Lokalisasi akar masalah	GNN	5 sampai 6	Relasi antarlayanan bersifat graf
Orkestrasi lintas tahap	Sistem multiagen	5 sampai 7	Terbukti pada MetaGPT, ChatDev, dan SWE-agent meningkatkan tingkat penyelesaian tugas

6.2 Berbasis Aturan: Validasi dan Gerbang Kepatuhan

Agen *Security-Policy* melakukan pemeriksaan deterministik sebelum aksi berisiko dijalankan. Pemeriksaan ini mengembalikan daftar pelanggaran, yaitu bidang wajib yang hilang, rahasia pada *diff* (bersifat pemblokir tanpa pengecualian), temuan SAST berat tinggi, dan *deployment* pada jendela pembekuan perubahan (*change freeze*). Pseudokodenya adalah sebagai berikut.

```
# Pseudokode Security-Policy: pemeriksaan deterministik sebelum aksi berisiko
```

```
def precheck(action, artifact, policy):
    violations = []
    if policy.require_fields and missing_fields(artifact,
policy.require_fields):
        violations.append("MANDATORY_FIELD_MISSING")
    if secret_scanner.scan(artifact.diff).found:
        violations.append("SECRET_IN_DIFF")          # blocking, tanpa
pengecualian
    if sast.scan(artifact.diff).severity >= "HIGH":
        violations.append("SAST_HIGH")
    if action == "deploy" and is_freeze_window(now()):
        violations.append("CHANGE_FREEZE")
    return violations
```

Aturan ini tidak menggunakan model probabilistik karena sifatnya adalah kepatuhan. Auditor harus dapat menelusuri alasan sebuah aksi ditolak.

6.3 Machine Learning Klasik: Klasifikasi Kategori dan Pengarahan

Metode ini memakai representasi TF-IDF atas judul dan deskripsi tiket, lalu mengklasifikasikannya dengan *Linear SVM* atau *Random Forest*. Bobot TF-IDF dihitung dengan rumus berikut, dengan tf(t,d) adalah frekuensi istilah t pada dokumen d, N adalah jumlah dokumen, dan df(t) adalah jumlah dokumen yang memuat istilah t.

$$TF - IDF(t,d) = tf(t,d) \times log\left(\frac{N}{1+df(t)}\right) \quad (14)$$

Fitur tambahan nonteks meliputi versi aplikasi, platform, jumlah pengguna terdampak, waktu kejadian, dan riwayat komponen. Metode ini dipilih untuk pengarahan kasar karena tiga alasan. Metode ini murah (inferensi kurang dari 5 ms), dapat diinterpretasi melalui bobot fitur, dan menjadi cadangan (*fallback*) ketika model DL tidak tersedia.

6.4 Deep Learning: Deteksi Duplikat dan Klasifikasi Severity

6.4.1 Deteksi Tiket Duplikat (Cosine Similarity atas Embedding)

Pendekatan ini mengikuti Isotani dkk. (2023) yang memakai *Sentence-BERT* untuk mengubah setiap elemen laporan *bug* menjadi vektor, lalu menghitung kemiripan semantik antarlaporan dari kemiripan vektornya. SBERT di-*fine-tune* memakai teks laporan *bug* agar sesuai dengan domain. Kemiripan dihitung dengan *cosine similarity* sebagai berikut.

$$cos(\theta) = \frac{A \cdot B}{\|A\| \, \|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\Sigma A_i^2} \cdot \sqrt{\Sigma B_i^2}} \quad (15)$$

Pada rumus tersebut, A dan B adalah vektor representasi (*embedding* atau TF-IDF) dari dua tiket yang dibandingkan, A_i dan B_i adalah komponen ke-i, dan n adalah jumlah dimensi vektor. Simbol ‖A‖ dan ‖B‖ adalah norma Euklides, dan θ adalah sudut antarvektor. Nilai cos(θ) yang mendekati 1 berarti sangat mirip (kandidat duplikat), sedangkan nilai yang mendekati 0 berarti tidak berkaitan.

Sebagai contoh, tiga tiket direpresentasikan dalam lima dimensi topik, yaitu *crash*, *login*, tombol antarmuka, lambat, dan *timeout*. Vektor ketiga tiket tersaji pada Tabel 23.

Tabel 23. Vektor fitur pada contoh perhitungan kemiripan tiket

Tiket	Deskripsi singkat	Vektor fitur
A	"Aplikasi *crash* saat *login*, koneksi *timeout*"	[0,80; 0,10; 0,00; 0,30; 0,20]
B	"*Login* gagal, aplikasi *force-close*, sering *timeout*"	[0,75; 0,15; 0,00; 0,35; 0,25]
C	"Tombol 'Simpan' di halaman profil tidak berfungsi"	[0,10; 0,00; 0,90; 0,05; 0,00]

Kemiripan tiket A dan B dihitung sebagai berikut. Hasilnya berada di atas ambang $\tau_{dup} = 0{,}85$ sehingga tiket B ditandai sebagai duplikat.

$$cos(A, B) = \frac{0{,}8 \cdot 0{,}75 + 0{,}1 \cdot 0{,}15 + 0 \cdot 0 + 0{,}3 \cdot 0{,}35 + 0{,}2 \cdot 0{,}25}{0{,}8832 \times 0{,}8775} = \frac{0{,}770}{0{,}7750} \approx 0,994 \quad (16)$$

Kemiripan tiket A dan C dihitung sebagai berikut. Hasilnya jauh di bawah ambang sehingga tiket C dianggap tiket baru.

$$cos(A, C) = \frac{0{,}8 \cdot 0{,}10 + 0{,}1 \cdot 0 + 0 \cdot 0{,}90 + 0{,}3 \cdot 0{,}05 + 0{,}2 \cdot 0}{0{,}8832 \times 0{,}9069} = \frac{0{,}095}{0{,}8010} \approx 0,119 \quad (17)$$

Tiket B tidak dibuatkan tiket investigasi baru, tetapi ditautkan ke tiket A. Inilah mekanisme yang secara langsung memotong beban kerja ulang sebesar 450 jam pada Subbab 2.3.

6.4.2 Klasifikasi Severity (Softmax)

Klasifikasi *severity* memakai fungsi *softmax* yang mengubah skor mentah (*logit*) menjadi probabilitas.

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \; j = 1 \ldots K \tag{18}$$

Pada rumus tersebut, z_k adalah skor mentah untuk kelas ke-k, dan K adalah jumlah kelas, yaitu empat (Low, Medium, High, dan Critical). Nilai P(y=k|x) adalah probabilitas bahwa tiket x tergolong kelas ke-k, dan seluruhnya berjumlah 1. Bilangan e ≈ 2,718 adalah basis logaritma natural sehingga seluruh skor bernilai positif dan proporsional.

Sebagai contoh, satu tiket masuk memiliki logit dan probabilitas seperti tersaji pada Tabel 24.

Tabel 24. Contoh perhitungan *softmax* untuk satu tiket

Kelas *severity*	Logit z	e^z	Probabilitas
Low	1,2	3,320	22,9 persen
Medium	0,5	1,649	11,4 persen
High	2,1	8,166	56,4 persen
Critical	0,3	1,350	9,3 persen
Jumlah		14,485	100 persen

Kelas terpilih adalah High sehingga tiket diarahkan ke antrean prioritas tinggi. Namun, karena P = 0,564 berada di bawah ambang keyakinan τ_High = 0,70, sistem mengeskalasi keputusan ke titik pemeriksaan manusia alih-alih memutuskan sendiri (lihat Subbab 6.8 dan Gambar 2).

Dasar metodologis prediksi *severity* mengikuti Ramay dkk. (2019). Mereka memproses teks laporan *bug* dengan NLP, menghitung skor emosi setiap laporan, menyusun vektor dari laporan yang telah diproses, lalu meneruskannya ke pengklasifikasi *deep neural network*. Untuk varian deteksi duplikat berbasis CNN, He dkk. (2020) merepresentasikan pasangan laporan sebagai matriks dua kanal dan melaporkan akurasi 0,9429 (OpenOffice), 0,9685 (Eclipse), 0,9534 (NetBeans), dan 0,9552 pada *dataset* gabungan.

6.5 Reinforcement Learning

RL dipakai pada dua titik keputusan berurutan yang umpan baliknya tertunda, yaitu prioritisasi kasus uji oleh *Worker-QA* dan kebijakan rilis *canary* oleh *Worker-Deploy*.

6.5.1 Prioritisasi Kasus Uji (Worker-QA)

Metode ini mengacu pada Retecs (Spieker dkk.). RL digunakan untuk memilih dan memprioritaskan kasus uji berdasarkan durasi, waktu eksekusi terakhir, dan riwayat kegagalan. Pada lingkungan yang terus berubah, yaitu kasus uji baru dibuat dan kasus uji usang dihapus, agen belajar menempatkan kasus uji yang rawan gagal pada posisi lebih tinggi melalui panduan fungsi *reward* dan pengamatan siklus CI sebelumnya. Tujuannya adalah meminimalkan waktu bolak-balik antara *commit* dan umpan balik kegagalan kepada pengembang.

Formulasi proses keputusan Markov (MDP) untuk masalah ini adalah sebagai berikut. Keadaan s terdiri atas fitur *diff* kode, riwayat kegagalan tes, durasi tes, waktu sejak eksekusi terakhir, dan status *flaky*. Aksi a adalah peringkat atau pemilihan subset tes dengan anggaran waktu T. *Reward* r dihitung sebagai berikut.

$$r = \Sigma(tes\ gagal\ yang\ ditemukan \times bobot\ posisi) - \lambda \cdot waktu_{eksekusi} \quad (19)$$

6.5.2 Kebijakan Rilis Canary (Worker-Deploy)

Masalah ini diformulasikan sebagai berikut. Keadaan terdiri atas risk_score, ukuran *diff*, hasil QA, jam atau hari, dan riwayat *rollback* layanan. Aksi yang tersedia adalah *hold*, *canary* 5 persen, *canary* 25 persen, dan *full deploy*. *Reward* bernilai +1 apabila tidak ada pelanggaran SLO dalam 30 menit, −5 apabila terjadi *rollback*, dan −0,2 per jam penundaan sebagai biaya kesempatan. Pembaruan *Q-learning* dilakukan dengan rumus berikut.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \cdot max_{a'} Q(s',a') - Q(s,a)\right] \quad (20)$$

Sebagai contoh, diketahui α = 0,1, γ = 0,9, dan Q(s, *canary*_5%) = 0,50. Agen memilih *canary*, tidak terjadi pelanggaran SLO sehingga r = +1, dan nilai

maksimum keadaan berikutnya adalah max Q(s′,·) = 0,80. Perhitungannya adalah sebagai berikut.

$$Q \leftarrow 0,50 \ + \ 0,1 \times (1 \ + \ 0,9 \times 0,80 \ - \ 0,50) \ = \ 0,50 \ + \ 0,1 \times 1,22 \ = \ 0,622 \quad (21)$$

Nilai aksi *canary* 5 persen pada keadaan tersebut naik dari 0,500 menjadi 0,622. Setelah banyak episode, kebijakan belajar kapan boleh langsung *full deploy* dan kapan harus menahan. Catatan keamanan: RL hanya boleh mengusulkan, dan aksi *full deploy* tetap terkunci di balik persetujuan manusia sesuai Tabel 21. RL yang mengendalikan produksi tanpa gerbang merupakan anti-pola.

6.6 Graph Neural Network: Lokalisasi Akar Masalah

Arsitektur *microservices* secara alami membentuk graf pemanggilan. Literatur AIOps menunjukkan bahwa GNN cocok untuk memodelkan struktur dependensi layanan dan bernalar atas data relasional. Dengan bekerja langsung pada graf pemanggilan, GNN menangkap hubungan topologis antarlayanan dan merambatkan informasi sepanjang sisi dependensi. Sistem DiagFusion, misalnya, memakai GNN untuk mengidentifikasi instans akar masalah sekaligus menentukan tipe kegagalan dalam graf dependensi.

Aturan agregasi satu lapis (GraphSAGE-mean, disederhanakan) dan pembacaan skor per simpul adalah sebagai berikut.

$$h_v \ = \ \sigma(W \cdot (x_v \ + \ mean_{u \in N(v)} \, x_u)) \quad (22)$$

$$skor(v) \ = \ w \cdot h_v \quad (23)$$

Sebagai contoh, graf terdiri atas tiga simpul, yaitu api-gateway (A) memanggil order-service (B) yang memanggil payment-db (C). Fitur simpul x berupa z-score latensi dan z-score laju galat pada jendela insiden, seperti tersaji pada Tabel 25.

Tabel 25. Fitur simpul pada contoh GNN

Simpul	Fitur x = [z-score latensi; z-score laju galat]
A (api-gateway)	[2,1; 1,0]

Simpul	Fitur x = [z-score latensi; z-score laju galat]
B (order-service)	[2,6; 1,2]
C (payment-db)	[3,4; 2,8]

Dengan W = 0,5·I dan σ berupa fungsi identitas, agregasi menghasilkan tiga vektor berikut.

$$h_A = 0,5 \times (2,1;\ 1,0 + 2,6;\ 1,2) = 2,350;\ 1,100$$

$$h_B = 0,5 \times (2,6;\ 1,2 + 2,75;\ 1,90) = 2,675;\ 1,550$$

$$h_C = 0,5 \times (3,4;\ 2,8 + 2,6;\ 1,2) = 3,000;\ 2,000$$

Nilai [2,75; 1,90] pada baris h_B adalah rata-rata fitur tetangga B, yaitu A dan C. Dengan pembacaan skor w = [0,6; 0,4], skor setiap simpul adalah sebagai berikut.

$$skor(A) = 0,6 \times 2,350 + 0,4 \times 1,100 = 1,850$$

$$skor(B) = 0,6 \times 2,675 + 0,4 \times 1,550 = 2,225$$

$$skor(C) = 0,6 \times 3,000 + 0,4 \times 2,000 = 2,600$$

Peringkat akar masalah adalah C (payment-db), lalu B, kemudian A. Hasil ini sejalan dengan intuisi SRE bahwa anomali di hulu sering merupakan efek rambatan dari komponen di hilir. Keluaran GNN berupa peringkat, bukan vonis. Selanjutnya, *Worker-Investigate* meminta LLM menyusun hipotesis dan langkah reproduksi berdasarkan bukti dari *Scout-Log*.

6.7 Generative AI dan LLM

LLM dipakai untuk empat tugas yang membutuhkan bahasa alami atau kode. Setiap tugas disertai pola *prompt* dan pengaman, seperti tersaji pada Tabel 26.

Tabel 26. Tugas LLM, pola *prompt*, dan pengamannya

Tugas	Pola *prompt*	Pengaman
Ringkasan tiket untuk pemberi persetujuan	Meringkas, mengutip bukti, dan mencantumkan ketidakpastian	Wajib menyertakan tautan bukti; dilarang mengarang ID tiket
Draf kriteria penerimaan	Format Given/When/Then dari laporan investigasi	Validasi skema; ditolak bila tidak lengkap

Tugas	Pola *prompt*	Pengaman
Generasi *patch*	RAG atas repositori dan *test-first*	Wajib lulus tes; tidak boleh menyentuh berkas pada *denylist*
Catatan rilis	*Diff* dan tiket tertaut	Ditinjau manusia sebelum publikasi

Untuk menekan halusinasi, semua keluaran LLM yang menjadi dasar keputusan wajib berlandaskan artefak pada *blackboard* (RAG). Setiap klaim faktual dalam ringkasan wajib membawa rujukan ke ID bukti.

6.8 Aturan Keputusan Otonomi (Gate Rule)

Aturan gerbang menentukan apakah agen boleh bertindak sendiri atau harus mengeskalasi keputusan kepada manusia.

$$Aksi = \begin{cases} Otonom, \ jika\ P(y = k \mid x) \geq \tau\ dan\ risk(k) \leq R_{maks}; \\ Eskalasi\ ke\ manusia, \ jika\ se \end{cases} \quad (24)$$

- τ (tau) adalah ambang keyakinan minimum agar agen boleh bertindak tanpa persetujuan manusia.
- risk(k) adalah estimasi risiko atau dampak kelas k. *Severity* Critical selalu berisiko tinggi walaupun keyakinannya tinggi.
- R_{maks} adalah batas risiko maksimum yang masih boleh dieksekusi secara otomatis. Nilai inilah yang menentukan kolom "persetujuan manusia wajib bila" pada Tabel 21.

Pada kasus Subbab 6.4.2, P(High) = 0,564 kurang dari τ = 0,70 sehingga keputusan dieskalasi, meskipun High bukan kelas yang paling berisiko. Apabila P(High) = 0,92 dan risk(High) = 0,6 ≤ R_{maks} = 0,6, keputusan diambil secara otonom.

6.9 Pertimbangan Untung-Rugi Antarmetode

Setiap metode memiliki karakteristik akurasi, biaya inferensi, keterjelasan, dan kebutuhan data yang berbeda, seperti tersaji pada Tabel 27.

Tabel 27. Perbandingan karakteristik metode

Metode	Akurasi relatif	Biaya inferensi	Keterjelasan	Kebutuhan data
Berbasis aturan	Rendah sampai sedang	Sangat rendah	Sangat tinggi	Tidak perlu
ML klasik	Sedang	Rendah	Tinggi	Ribuan sampel
DL (SBERT/BERT)	Tinggi	Sedang (GPU opsional)	Rendah sampai sedang	Puluhan ribu sampel
RL	Kontekstual	Sedang (pelatihan mahal)	Rendah	Simulator atau riwayat siklus
GNN	Tinggi pada data relasional	Sedang	Sedang (dengan *explainer*)	Graf dan label insiden
LLM	Sangat tinggi pada bahasa dan kode	Tinggi (per token)	Rendah	*Zero-shot* atau *few-shot*

Implikasinya jelas. Menempatkan LLM pada tahap triase bervolume tinggi merupakan pemborosan, sedangkan menempatkan metode berbasis aturan pada tahap generasi kode adalah kemustahilan. Arsitektur hibrida menempatkan setiap metode pada tahap yang sesuai.

BAB 7

BASIS DATA DAN DATASET

Bab ini menjelaskan tempat data disimpan dan data yang dipakai untuk melatih serta menguji model. Pembahasan dimulai dari basis data operasional, dilanjutkan dengan basis data vektor, kemudian dataset pelatihan, integrasi antarmuka pemrograman aplikasi (API) simulasi, dan diakhiri dengan tata kelola data.

7.1 Basis Data Operasional

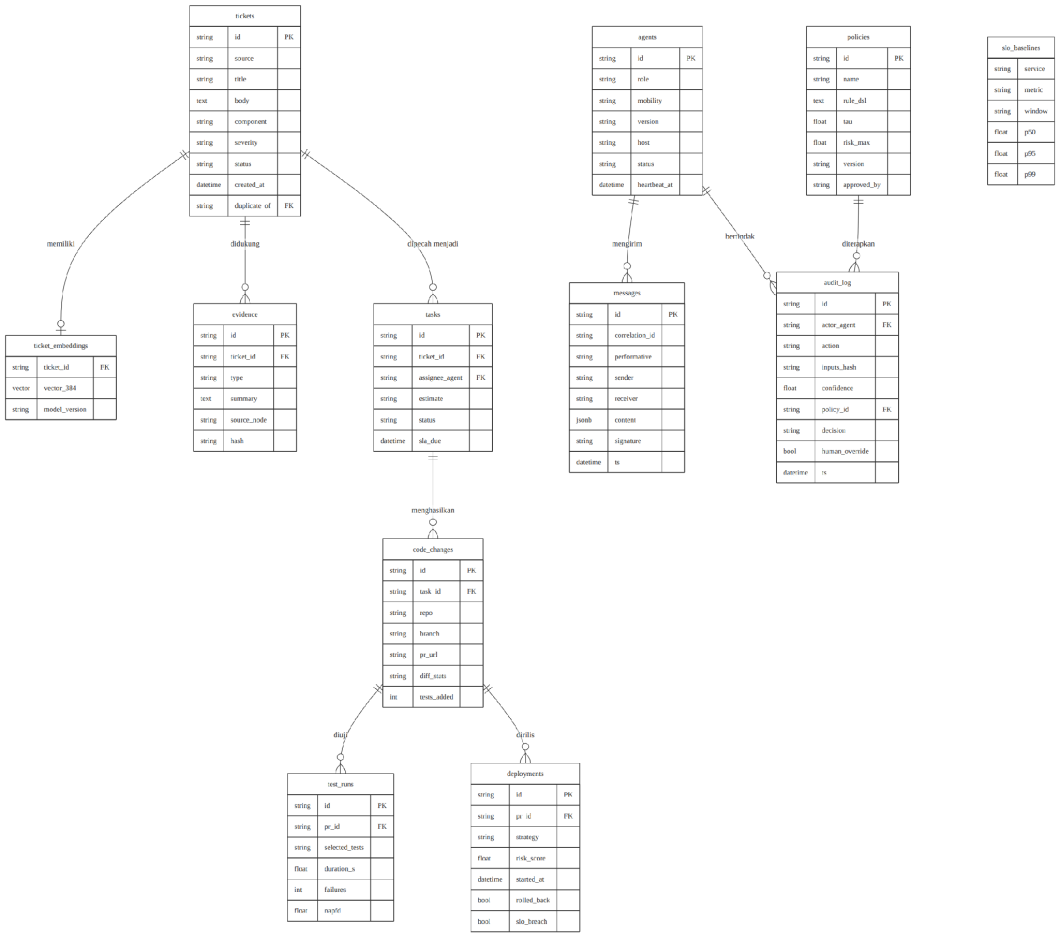
Basis data relasional PostgreSQL menyimpan keadaan transaksional alur kerja. Basis data ini sekaligus berfungsi sebagai papan tulis bersama (*blackboard*) yang dapat diaudit oleh semua agen. Rancangan terdiri atas dua belas tabel, yang tersaji pada Tabel 28, dan hubungan antartabel digambarkan pada Gambar 3.

Tabel 28. Tabel basis data operasional

Tabel	Kolom kunci	Fungsi
tickets	id, source, title, body, component, severity, status, created_at, duplicate_of	Entitas utama tiket
ticket_embeddings	ticket_id, vector(384), model_version	Rujukan ke basis data vektor; versi model dicatat demi reproduktibilitas
agents	id, role, mobility, version, host, status, heartbeat_at	Registri agen, termasuk posisi agen bergerak
messages	id, correlation_id, performative, sender, receiver, content (jsonb), signature, ts	Seluruh pesan antaragen
evidence	id, ticket_id, type (log/metric/trace), summary, source_node, hash	Bukti teragregasi dari *Scout-Log*
tasks	id, ticket_id, assignee_agent, estimate, status, sla_due	Hasil dekomposisi *Worker-Plan*
code_changes	id, task_id, repo, branch, pr_url, diff_stats, tests_added	Keluaran *Worker-Impl*
test_runs	id, pr_id, selected_tests, duration_s, failures, napfd	Keluaran *Worker-QA*
deployments	id, pr_id, strategy, risk_score, started_at, rolled_back, slo_breach	Keluaran *Worker-Deploy*
policies	id, name, rule_dsl, tau, risk_max, version, approved_by	Kebijakan tata kelola berversi

Tabel	Kolom kunci	Fungsi
audit_log	id, actor_agent, action, inputs_hash, confidence, policy_id, decision, human_override, ts	Jejak audit yang tidak dapat diubah (*append-only*)
slo_baselines	service, metric, window, p50, p95, p99	Garis dasar bagi *Scout-Monitor*

Tabel *audit_log* dirancang bersifat *append-only*. Artinya, sebuah pemicu (*trigger*) pada basis data menolak setiap perintah UPDATE dan DELETE terhadap tabel tersebut. Rancangan ini menjamin bahwa jejak keputusan agen tidak dapat diubah setelah dicatat.



Gambar 3. Diagram relasi entitas basis data T2D-MAS

7.2 Basis Data Vektor

Basis data vektor menyimpan representasi semantik tiket sehingga pencarian tiket serupa dapat dilakukan dengan cepat. Pilihan rancangannya tersaji pada Tabel 29.

Tabel 29. Rancangan basis data vektor

Aspek	Pilihan	Alasan
Mesin	FAISS (lokal, prototipe), kemudian Qdrant atau pgvector (produksi)	FAISS cukup untuk paling banyak satu juta vektor tanpa peladen tambahan
Dimensi	384 (all-MiniLM-L6-v2) atau 768 (SBERT base)	Kompromi antara kecepatan dan akurasi
Metrik	Hasil kali dalam atas vektor ternormalisasi, yang setara dengan kemiripan kosinus	Konsisten dengan rumus pada Subbab 6.4
Indeks	IndexFlatIP (prototipe), kemudian HNSW (skala besar)	Pencarian eksak memudahkan verifikasi angka pada laporan
Kebijakan versi	Pengindeksan ulang wajib dilakukan saat model_version berubah	Mencegah pembandingan vektor dari dua model yang berbeda

7.3 Dataset untuk Pelatihan ML dan DL

Tugas menyebut dataset publik seperti Kaggle sebagai contoh. Untuk domain *ticket-to-deployment*, kami memilih dataset pada Tabel 30 karena skemanya paling mendekati tiket perusahaan. Kolom terakhir tabel memuat catatan penggunaan setiap dataset.

Tabel 30. Dataset yang dipertimbangkan

No.	Dataset	Isi	Dipakai untuk	Catatan
D1	GitBugs	Lebih dari 150.000 laporan bug dari sembilan proyek *open source* yang aktif dipelihara, antara lain Firefox, Cassandra, dan VS Code. Data diagregasi dari pelacak isu GitHub, Bugzilla, dan Jira, dengan kolom kategori terstandar dan pembagian latih/uji yang telah ditentukan untuk deteksi duplikat	Deteksi duplikat, triase, prediksi *severity*	Prioritas utama karena menyediakan pembagian baku sehingga hasil dapat dibandingkan
D2	Eclipse & Mozilla Defect Tracking Dataset (Lamkanfi dkk., MSR 2013)	Lebih dari 200.000 laporan bug dari proyek Eclipse dan Mozilla, lengkap dengan seluruh modifikasi inkremental sepanjang umur laporan; tersedia bebas untuk keperluan riset	Pelatihan *severity*, analisis waktu perbaikan bug	Riwayat modifikasi berguna untuk memodelkan dinamika status
D3	bughub / BugRepo (LogPAI)	Kumpulan laporan bug teks bebas untuk identifikasi isu duplikat dari proyek seperti Firefox dan Eclipse JDT	*Benchmark* NLP untuk kemiripan tekstual	Acuan umum dalam literatur duplikat
D4	Dataset duplikat Eclipse, Firefox, dan Mobile 2017–2022	Rasio duplikat terukur: 6,60% (Eclipse), 20,53% (Firefox), dan 10,56% (Mobile)	Kalibrasi laju dasar duplikat dan ambang τ	Penting agar metrik tidak bias oleh kelas yang tidak seimbang

No.	Dataset	Isi	Dipakai untuk	Catatan
D5	SEOSS 33	Kebutuhan, laporan bug, riwayat kode, dan tautan jejak untuk 33 proyek utuh	Menghubungkan tiket dengan perubahan kode (bagi *Worker-Impl*)	Berguna untuk percobaan ketertelusuran
D6	SWE-bench dan SWE-bench Verified	500 isu GitHub nyata hasil kurasi manusia dari repositori Python populer; agen menerima isu dan keadaan repositori, lalu menghasilkan *patch* yang dinilai melalui eksekusi tes sungguhan	Evaluasi *Worker-Impl*	Skor mutakhir mendekati titik jenuh sehingga dipakai sebagai uji kewajaran, bukan klaim utama; audit menemukan sebagian kasus uji bermasalah
D7	Dataset internal tersimulasi	Tiket sintetis yang dibangkitkan oleh generator berbenih tetap dengan skema perusahaan (versi *synthetic-eval-v1*)	Demo menyeluruh, UI, dan seluruh percobaan A0–A3 yang dilaporkan	Dataset inilah yang benar-benar dipakai pada evaluasi (lihat Subbab 7.3.2)
D8	Dataset Kaggle pilihan tim	Belum dipilih	Opsional, apabila dosen mensyaratkan Kaggle secara eksplisit	Tidak dipakai pada versi laporan ini; kata kunci pencarian: *bug report*, *customer support tickets*, *IT service desk*

Perlu ditegaskan bahwa repositori kami menyediakan pemuat data (*loader*) untuk GitBugs dan Eclipse/Mozilla. Namun, percobaan A0–A3 yang dilaporkan pada Bab 9 memakai dataset sintetis D7, bukan dataset publik.

7.3.1 Pembagian dan Praproses

Praproses dan pembagian data mengikuti empat aturan berikut.

1. **Pembagian.** Data dibagi menjadi data latih 70 persen, data validasi 10 persen, dan data uji 20 persen, dengan stratifikasi menurut *severity*.
2. **Pemisahan waktu.** Pembagian dilakukan secara kronologis, yaitu tiket lama menjadi data latih dan tiket baru menjadi data uji. Aturan ini mencegah kebocoran informasi masa depan.
3. **Praproses teks.** Huruf kecil (opsional), penghapusan baris *stack trace* yang berulang, normalisasi versi dan nomor *build*, serta penyamaran (*masking*) PII seperti surel, nomor telepon, dan token.

4. **Ketidakseimbangan kelas.** Bobot kelas atau *focal loss* dipakai untuk kelas *Critical* yang jarang muncul.

7.3.2 Metrik Dataset

Metrik dataset perlu dilaporkan agar pembaca dapat menilai kelayakan data. Pada versi laporan ini, kami hanya dapat menyajikan fakta yang tercatat pada berkas konfigurasi percobaan. Nilai lain, seperti rasio duplikat aktual dan distribusi *severity* pada dataset publik, belum diukur karena dataset publik belum dipakai pada percobaan. Kondisi tersebut tersaji pada Tabel 31.

Tabel 31. Metrik dataset pada percobaan yang dilaporkan

Metrik	Nilai	Keterangan
Versi dataset	synthetic-eval-v1	Dicatat pada berkas konfigurasi percobaan
Jumlah tiket per konfigurasi	500 tiket	100 tiket sintetis × 5 benih (42 sampai 46)
Kolom tiket	ID, judul, isi, komponen, *severity*, waktu pembuatan, tautan duplikat, label kesalahan	Kasus yang sama dipakai oleh keempat konfigurasi
Rasio duplikat pada dataset publik	Belum diukur	Perlu dibandingkan dengan D4 setelah dataset publik dipakai
Distribusi *severity* pada dataset publik	Belum diukur	Perlu disajikan dengan diagram batang
Panjang deskripsi (median dan persentil ke-95)	Belum diukur	Menentukan nilai max_length pada *tokenizer*

7.4 Integrasi API Perusahaan (Simulasi)

Karena akses ke sistem berbayar tidak tersedia, integrasi disimulasikan melalui layanan tiruan (*mock service*) yang meniru bentuk respons sistem nyata. Layanan tiruan tersebut dibuat dengan FastAPI, dan setiap layanan dirancang memiliki latensi acak 50 sampai 300 milidetik serta tingkat galat 2 persen agar agen terlatih menangani kegagalan. Pemetaannya tersaji pada Tabel 32.

Tabel 32. Layanan tiruan dan titik akhir yang ditiru

Sistem nyata	Layanan tiruan	Titik akhir yang ditiru
Jira REST API	mock_jira (FastAPI)	GET /issue/{key}, POST /issue, PUT /issue/{key}/transitions

Sistem nyata	Layanan tiruan	Titik akhir yang ditiru
GitHub API	mock_github	POST /repos/{o}/{r}/pulls, GET /commits, POST /check-runs
Datadog atau observabilitas	mock_obs	GET /logs/query, GET /metrics/query, GET /traces
CI/CD	mock_ci	POST /pipelines/trigger, GET /pipelines/{id}
Infrastruktur dan *deploy*	mock_infra	POST /deploy, POST /rollback, GET /slo

Berikut ini contoh muatan (*payload*) tiket ternormalisasi yang ditulis oleh *Scout-Feedback* ke papan tulis bersama.

```
{
  "ticket_id": "TCK-1042",
  "source": "mobile_crash_reporter",
  "title": "Checkout gagal saat flash sale, aplikasi force close",
  "body": "Saat menekan Bayar pada pukul 20:03 aplikasi menutup sendiri...",
  "app_version": "4.18.2",
  "platform": "android-14",
  "affected_users_estimate": 2143,
  "attachments": ["stacktrace_01.txt"],
  "pii_masked": true,
  "created_at": "2026-09-14T20:05:11+07:00"
}
```

7.5 Tata Kelola Data

Empat aturan tata kelola data pada Tabel 33 berlaku untuk seluruh komponen sistem.

Tabel 33. Aturan tata kelola data

Aspek	Aturan
PII	*Scout-Log* menyamarkan PII sebelum data meninggalkan simpul produksi
Retensi	Log mentah tidak disalin; hanya ringkasan dan *hash* bukti yang disimpan
Lisensi dataset	Dataset Eclipse/Mozilla tersedia bebas untuk keperluan riset; sitasinya dicantumkan pada Bab 11 dan pada berkas README repositori
Reproduktibilitas	dataset_version, model_version, dan benih (*seed*) disimpan pada setiap percobaan

BAB 8

PURWARUPA DAN FITUR

8.1 Lingkup Purwarupa

Purwarupa T2D-MAS dibangun sebagai bukti konsep (*proof of concept*) dalam lingkup mata kuliah. Tabel 34 menyajikan status setiap komponen. Kolom status telah kami sesuaikan dengan kondisi kode terakhir sehingga berbeda dari rancangan awal pada satu hal penting, yaitu model bahasa besar belum terhubung.

Tabel 34. Lingkup dan status purwarupa

Komponen	Status pada purwarupa
Empat arketipe agen (11 instansiasi)	Diimplementasikan sebagai proses Python terpisah yang berkomunikasi melalui *message bus*
Mobilitas agen	Disimulasikan dengan kontainer Docker sebagai simpul. Agen dikirim sebagai paket berisi kode dan keadaan, ditandatangani dengan Ed25519, lalu dijalankan pada kontainer tujuan
Model bahasa besar (LLM)	Belum terhubung. Sandbox dan harness evaluasi tidak memanggil LLM daring. Tugas LLM yang mungkin dilakukan pada masa depan hanya didokumentasikan pada repositori
Dataset	Pemuat data publik tersedia, tetapi percobaan yang dilaporkan memakai tiket sintetis (Subbab 7.3)
Deployment nyata	Tidak ada. Hanya layanan infrastruktur tiruan yang dipakai
Alur menyeluruh	Belum berjalan otomatis dari awal sampai akhir. Sandbox menjalankan normalisasi masukan dan triase berbasis aturan, lalu menampilkan jeda persetujuan dan penanda tahap berikutnya. Modul RCA, RL, eksekusi bergerak, dan *benchmark* berada pada jalur kode yang terpisah

8.2 Tumpukan Teknologi

Tabel 35 memuat teknologi yang dipilih untuk setiap lapisan beserta alasannya.

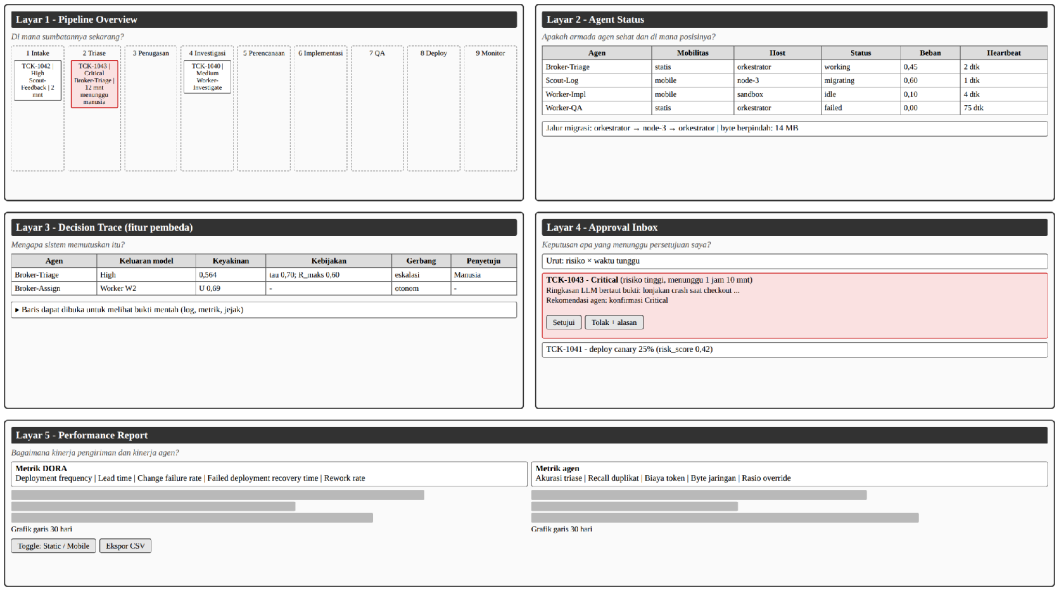
Tabel 35. Tumpukan teknologi

Lapisan	Pilihan	Alasan
Bahasa	Python 3.11	Ekosistem ML paling lengkap
Orkestrasi agen	LangGraph atau mesin keadaan buatan sendiri	Kendali eksplisit atas transisi dan gerbang
Message bus	Redis Streams	Ringan dan mendukung *consumer group* untuk paralelisasi *Worker*

Lapisan	Pilihan	Alasan
Model teks	sentence-transformers dan HuggingFace Transformers	Sesuai Subbab 6.4
RL	Gymnasium dan stable-baselines3	Standar dan mudah direproduksi
GNN	PyTorch Geometric	Sesuai Subbab 6.6
Basis data	PostgreSQL dengan pgvector atau FAISS	Sesuai Bab 7
API *backend*	FastAPI	Cepat dan menghasilkan dokumentasi OpenAPI otomatis
Frontend	React, Tailwind, dan Recharts	Dasbor interaktif
Kontainer	Docker Compose	Menyimulasikan banyak simpul bagi agen bergerak

8.3 Rancangan UI/UX

Prinsip rancangan antarmuka kami adalah satu layar menjawab satu pertanyaan. Dasbor tidak boleh menuntut pengguna menebak pekerjaan yang sedang dikerjakan agen. Kerangka kelima layar tersaji pada Gambar 4, dan penjelasan setiap layar diuraikan setelahnya.



Gambar 4. Kerangka (*wireframe*) dasbor T2D-MAS

8.3.1 Layar 1: Pipeline Overview

Layar ini menampilkan papan Kanban dengan sembilan kolom sesuai tahap alur kerja. Setiap kartu tiket menampilkan ID, warna *severity*, agen yang sedang memegang tiket, waktu di tahap saat ini, dan ikon peringatan apabila tiket

menunggu manusia. Layar ini menjawab pertanyaan, “Di mana sumbatannya sekarang?”

8.3.2 Layar 2: Agent Status

Layar ini menyajikan tabel atau kartu seluruh agen. Setiap baris memuat peran, mobilitas (ikon pin untuk agen statis dan ikon pesawat untuk agen bergerak), *host* saat ini, status (*idle*, *working*, *migrating*, atau *failed*), beban, dan denyut terakhir. Untuk agen bergerak, layar menampilkan jalur migrasi (orkestrator, simpul 3, lalu kembali ke orkestrator) beserta jumlah byte yang berpindah. Layar ini menjawab pertanyaan, “Apakah armada agen sehat, dan di mana posisinya?”

8.3.3 Layar 3: Decision Trace (Fitur Pembeda)

Untuk satu tiket, layar ini menampilkan linimasa setiap keputusan. Setiap entri memuat agen, masukan, keluaran model, nilai keyakinan, kebijakan yang berlaku (τ dan R_{maks}), hasil gerbang (otonom atau eskalasi), dan pihak yang menyetujui. Pengguna dapat membuka setiap baris untuk melihat bukti mentah. Layar ini menjawab pertanyaan, “Mengapa sistem memutuskan hal itu?” Kemampuan tersebut membuat sistem dapat diaudit dan merupakan hal yang paling sering absen pada produk komersial.

8.3.4 Layar 4: Approval Inbox

Layar ini memuat antrean keputusan yang dieskalasi. Antrean diurutkan menurut hasil kali risiko dan waktu tunggu. Setiap butir menyajikan ringkasan yang berlandaskan bukti, rekomendasi agen, serta dua tombol, yaitu Setujui dan Tolak disertai alasan. Alasan penolakan masuk ke data pelatihan ulang.

8.3.5 Layar 5: Performance Report

Layar ini menampilkan metrik DORA berdampingan dengan metrik agen. Metrik DORA meliputi frekuensi *deployment*, waktu tunggu perubahan (*lead time*), tingkat kegagalan perubahan, waktu pemulihan dari *deployment* gagal, dan tingkat kerja ulang. Metrik agen meliputi akurasi triase, *recall* duplikat, biaya token, byte jaringan, dan tingkat *override*. Data dapat diekspor ke berkas CSV.

8.3.6 Aturan UX

- Setiap aksi otonom dapat dibatalkan dalam 30 detik (jendela pembatalan).
- Warna tidak pernah menjadi satu-satunya pembawa makna, demi aksesibilitas.
- Status agen diperbarui melalui *server-sent events*, bukan *polling*, agar linimasa terasa hidup.

8.4 Fitur Unggulan

Lima fitur unggulan tersaji pada Tabel 36 beserta cara mendemonstrasikannya.

Tabel 36. Fitur unggulan dan cara demonstrasi

Fitur	Deskripsi	Cara demonstrasi
Migrasi agen	*Scout-Log* dikirim ke enam kontainer simpul, memproses log lokal, dan mengembalikan vektor fitur	Tombol “Investigate”, lalu animasi jalur migrasi dan penghitung byte
Laporan performa	Perbandingan langsung mode statis dan mode bergerak pada tiket yang sama	Sakelar “Static / Mobile”, lalu bandingkan waktu dan byte
Keamanan berlapis	(1) verifikasi tanda tangan agen, (2) *sandbox* cgroup, (3) daftar izin *egress*, (4) pemindaian rahasia pada diff, (5) gerbang otonomi, dan (6) audit *append-only*	Uji injeksi: sisipkan rahasia palsu pada diff, lalu tunjukkan penolakannya
Koreksi manusia (*human override*)	Setiap keputusan dapat dikoreksi, dan koreksi menjadi label pelatihan	Ubah *severity* di Approval Inbox, lalu tunjukkan entri audit
Simulasi kasus nyata	Skenario insiden *flash sale* dari awal sampai akhir	Lihat Subbab 8.5

8.5 Simulasi Kasus Nyata: Insiden Flash Sale

Untuk menunjukkan alur kerja secara utuh, kami menyusun skenario insiden pada saat *flash sale*. Tabel 37 mencatat setiap peristiwa berdasarkan waktu relatif terhadap kejadian pertama (T+). Angka pada tabel merupakan hasil perhitungan rancangan dan simulasi, bukan hasil pengukuran pada sistem produksi.

Tabel 37. Linimasa simulasi insiden *flash sale*

Waktu (T+)	Peristiwa	Agen	Keluaran
00:00	2.143 laporan *crash* masuk dalam 4 menit	*Scout-Feedback*	Satu tiket induk TCK-1042 dan 2.142 tiket tertaut
00:12	Pencarian vektor menemukan TCK-0987 dengan cos = 0,91	*Broker-Triage*	Ditandai berkaitan, bukan duplikat penuh, karena konteks beban berbeda
00:20	*Softmax* menghasilkan P(*Critical*) = 0,88 dan risk(*Critical*) = 1,0 > R_{maks}	*Broker-Triage* dan *Security-Policy*	Eskalasi ke manusia (sesuai Subbab 6.8)
01:30	Manusia mengonfirmasi *Critical*	*Approval Inbox*	Status berubah dan SLA 2 jam aktif
01:35	Lelang CNP dimenangkan W2 (U = 0,69)	*Broker-Assign*	*Worker-Investigate* #2 ditetapkan
01:40	*Scout-Log* bermigrasi ke enam simpul dan memproses 8,19 GB log secara lokal	*Scout-Log* (bergerak)	84 MB berpindah melalui jaringan dalam 27 detik (dibandingkan ±11 menit pada mode statis)
02:05	GNN menempatkan payment-db sebagai tersangka utama (skor 2,60)	*Worker-Investigate*	Hipotesis: kehabisan kolam koneksi saat lonjakan
02:20	Dekomposisi: naikkan kolam koneksi, tambah *circuit breaker*, dan uji beban	*Worker-Plan*	Tiga subtugas beserta kriteria penerimaan
02:50	*Patch* dan tes dibuat dalam *sandbox* efemeral	*Worker-Impl* (bergerak)	Draf PR dan dua tes baru
03:05	RL memilih 120 dari 3.400 tes (anggaran 8 menit), semuanya lulus	*Worker-QA*	NAPFD 0,94 (asumsi)
03:15	Tinjauan kode oleh manusia, lalu penggabungan	Manusia	Gerbang ke-2 terlewati
03:25	risk_score = 0,28, sehingga *canary* 5% berjalan otonom	*Worker-Deploy*	Tidak ada pelanggaran SLO dalam 30 menit
04:00	Persetujuan rilis penuh	Manusia	Gerbang ke-3 terlewati
04:30	Garis dasar SLO diperbarui dan tiket ditutup	*Scout-Monitor*	Umpan balik ke *Scout-Feedback*

Waktu pemulihan (MTTR) pada simulasi ini adalah 4 jam 30 menit. Garis dasar manual belum diukur sehingga perbandingan langsung belum dapat disajikan. Perlu dicermati bahwa dua dari tiga penundaan terbesar berasal dari penantian terhadap manusia, yaitu pada T+00:20 sampai 01:30 dan pada T+03:15. Temuan ini penting bagi Bab 10 karena menunjukkan bahwa setelah otomasi, sumbatan bergeser ke ketersediaan pemberi persetujuan, bukan ke kemampuan teknis agen.

BAB 9

EVALUASI

9.1 Desain Eksperimen

Evaluasi membandingkan empat konfigurasi (*arm*) pada beban kerja yang identik, yaitu tiket dan benih yang sama. Perlu ditegaskan bahwa A0 sampai A3 adalah konfigurasi *benchmark*, bukan empat peran agen. Keempat konfigurasi tersaji pada Tabel 38.

Tabel 38. Konfigurasi percobaan

Konfigurasi	Deskripsi	Fungsi
A0: manual atau berbasis aturan	Triase kata kunci dan proses manual, dalam satu proses tanpa pesan agen	Garis dasar "sistem statis" yang diminta tugas
A1: agen tunggal	Satu agen menangani seluruh alur	Menguji apakah multiagen benar-benar diperlukan
A2: multiagen statis	Beberapa agen statis; seluruhnya berjalan terpusat dan bertukar pesan antarperan	Mengisolasi kontribusi spesialisasi
A3: multiagen dan bergerak	Konfigurasi penuh T2D-MAS; paket bertanda tangan dijalankan pada simpul data	Mengisolasi kontribusi mobilitas

Perbandingan A2 dengan A3 menjadi inti klaim bahwa agen bergerak lebih baik daripada agen statis. Adapun perbandingan A1 dengan A2 menjadi inti klaim bahwa multiagen lebih baik daripada agen tunggal. Tanpa kedua perbandingan ini, klaim keunggulan tidak dapat dipertahankan.

Protokol yang dirancang. Rancangan awal menetapkan 30 tiket per konfigurasi dengan lima pengulangan (benih berbeda), sehingga menghasilkan 150 lintasan per konfigurasi. Perangkat keras dan versi model dikunci, dan hasil dilaporkan sebagai median dan rentang antarkuartil.

Protokol yang dijalankan. Percobaan yang benar-benar dijalankan memakai 100 tiket sintetis dengan lima benih (42 sampai 46), sehingga setiap konfigurasi memproses 500 tiket. Kasus yang sama dipakai pada keempat konfigurasi. Kami memilih melaporkan protokol yang dijalankan pada Subbab 9.4 dan 9.5.

9.2 Metrik

9.2.1 Metrik yang Diminta Tugas

Tugas meminta empat indikator, yang definisi operasionalnya tersaji pada Tabel 39.

Tabel 39. Definisi operasional indikator evaluasi

Indikator	Definisi operasional	Satuan
Waktu eksekusi	Waktu dinding dari tiket masuk sampai berstatus *deployed*; waktu sampai eskalasi pertama dilaporkan terpisah	detik
Biaya komunikasi	Total byte yang melintasi jaringan, ditambah jumlah pesan antaragen dan jumlah token LLM	MB, pesan, token
Tingkat keamanan	Jumlah pelanggaran kebijakan yang lolos ke tahap berikutnya dibagi jumlah pelanggaran yang disuntikkan	rasio (target 0)
Efisiensi sumber daya	CPU-detik, puncak memori, dan biaya inferensi	CPU·s, MB, USD

9.2.2 Metrik Mutu Model

Mutu model dinilai dengan empat metrik berikut. *Recall@k* mengukur proporsi duplikat benar yang muncul pada k peringkat teratas.

$$Recall@k = \frac{|\{duplikat\ benar\ pada\ k\ teratas\}|}{|\{duplikat\ benar\}|} \tag{25}$$

Mean Average Precision (MAP) merata-ratakan presisi rata-rata pada seluruh kueri Q, dengan m_i menyatakan jumlah dokumen relevan pada kueri ke-i dan R_{ij} menyatakan peringkat dokumen relevan ke-j pada kueri tersebut.

$$MAP = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{m_i}\sum_{j=1}^{m_i} Precision(R_{ij}) \tag{26}$$

Macro-F1 merata-ratakan F1 setiap kelas sebanyak K kelas tanpa memberi bobot berbeda pada kelas yang jarang.

$$F1_{macro} = \frac{1}{K}\sum_{k=1}^{K}\frac{2 \times P_k \times R_k}{P_k + R_k} \tag{27}$$

NAPFD (*Normalized Average Percentage of Faults Detected*) menilai kualitas prioritisasi tes, yaitu seberapa awal kegagalan terdeteksi dalam urutan tes

yang dipilih. Nilainya berkisar dari 0 sampai 1, dan nilai yang lebih tinggi menunjukkan bahwa kegagalan terdeteksi lebih awal.

9.2.3 Metrik Penyampaian (DORA)

Kerangka DORA versi terbaru mengelompokkan metrik menjadi dua kelompok. Kelompok *throughput* terdiri atas frekuensi *deployment*, waktu tunggu perubahan, dan waktu pemulihan dari *deployment* gagal. Kelompok *instability* terdiri atas tingkat kegagalan perubahan dan tingkat kerja ulang. Keandalan menjadi metrik pelengkap yang berbasis SLO dan SLI. Semua metrik ini dilaporkan pada layar Performance Report.

Peringatan interpretasi. Riset tahun 2025 sampai 2026 menemukan bahwa adopsi AI meningkatkan *throughput*, tetapi sering disertai memburuknya stabilitas penyampaian. Frekuensi *deployment* dan waktu tunggu membaik, sedangkan tingkat kegagalan perubahan dapat memburuk. Oleh karena itu, kenaikan kecepatan tidak boleh dilaporkan tanpa melaporkan stabilitas pada baris yang sama. Alasan inilah yang menjadikan tingkat kerja ulang sebagai salah satu metrik.

9.3 Rumus Perbandingan

Lima rumus berikut dipakai untuk membandingkan konfigurasi. Rumus pertama menghitung percepatan A3 terhadap A0.

$$Speedup = \frac{T_{A0}}{T_{A3}} \tag{28}$$

Rumus kedua menghitung penurunan komunikasi A3 terhadap A2.

$$Reduksi\ komunikasi = \frac{Byte_{A2} - Byte_{A3}}{Byte_{A2}} \times 100\% \tag{29}$$

Rumus ketiga menghitung efisiensi biaya A3 terhadap A0.

$$Efisiensi\ biaya = \frac{Biaya_{A0} - Biaya_{A3}}{Biaya_{A0}} \times 100\% \tag{30}$$

Rumus keempat menghitung tingkat otonomi, dan rumus kelima menghitung tingkat *override* manusia.

$$Tingkat\ otonomi\ =\ \frac{keputusan\ otonom}{total\ keputusan} \quad (31)$$

$$Tingkat\ override\ =\ \frac{keputusan\ otonom\ yang\ dibatalkan\ manusia}{keputusan\ otonom} \quad (32)$$

9.4 Hasil Percobaan

Subbab ini menyajikan hasil yang benar-benar tercatat. Angka bersumber dari berkas ringkasan percobaan bertanda waktu 30 September 2026 (hasil observasi waktu pada *host*, simulasi sintetis, atau estimasi). Tabel 40 memuat hasil ringkasan dari 500 tiket sintetis pada setiap konfigurasi.

Tabel 40. Hasil percobaan pada 500 tiket sintetis per konfigurasi

Metrik	A0	A1	A2	A3
Rata-rata waktu dinding per tiket	0,102 ms	0,155 ms	0,184 ms	0,198 ms
Rata-rata byte jaringan per tiket	0	1.778	3.556	4.161
Rata-rata pesan antaragen per tiket	0	6	12	13
Macro-F1 severity	1,00	1,00	1,00	1,00
NAPFD prioritisasi kesalahan	0,49	0,75	0,75	0,75
Laju *rollback*	5,6%	5,4%	5,8%	7,4%
Rilis penuh tanpa persetujuan	0	0	0	0

Hasil pada Tabel 40 dapat dibaca melalui lima temuan berikut.

1. **Mutu klasifikasi sama.** Keempat konfigurasi mencapai *macro-F1* 1,00 pada tugas sintetis ini. Nilai tersebut menunjukkan bahwa tugas sintetis terlalu mudah untuk membedakan konfigurasi, dan tidak dapat diartikan sebagai kinerja pada tiket nyata.
2. **Prioritisasi kesalahan membaik.** A1, A2, dan A3 mencapai NAPFD 0,75, sedangkan A0 hanya 0,49. Dengan demikian, ketiga konfigurasi berbasis agen memprioritaskan kesalahan lebih baik daripada A0.
3. **Koordinasi menambah biaya.** Waktu dinding dan jumlah pesan meningkat dari A0 ke A3. A2 dan A3 menambah beban komunikasi, dan A3 tidak otomatis lebih baik karena jalur agen bergerak menambah biaya pengiriman dan waktu jalan.
4. **Byte A3 lebih besar daripada A2.** Pada percobaan ini, A3 mengirim 4.161 byte per tiket dibandingkan 3.556 byte pada A2. Hasil ini berbeda

dengan proyeksi rancangan pada Subbab 5.4 (penurunan sekitar 98,97 persen) karena percobaan memakai muatan kecil, bukan log berukuran gigabita. Manfaat kelokalan data pada A3 bersifat arsitektural dan belum diukur oleh percobaan ini.

5. **Invarian keamanan terjaga.** Tidak ada rilis penuh tanpa persetujuan pada semua konfigurasi. A3 melakukan 11 panggilan rilis penuh yang telah disetujui pada simulasi.

Selanjutnya, Tabel 41 membandingkan target rancangan dengan status pengukurannya. Metrik yang belum diukur ditandai jelas agar tidak disalahartikan sebagai hasil.

Tabel 41. Target rancangan dan status pengukuran

Metrik	Target	Status pada percobaan
Waktu triase persentil ke-95	≤ 30 detik	Belum diukur; hanya waktu dinding rata-rata yang tercatat (mikrodetik hingga milidetik)
Waktu tiket sampai *deploy*	≤ 60% dari A0	Belum diukur; tidak ada *deployment* nyata
Byte jaringan per investigasi	≤ 100 MB	Hanya proyeksi rancangan (8.190 MB pada A2, 84 MB pada A3); belum diukur
Token LLM per tiket	-	Berupa estimasi; tidak ada pemanggilan LLM
Recall@10 duplikat	≥ 0,80	Belum diukur pada data publik; percobaan memakai kedekatan token sebagai perkiraan
Macro-F1 severity	≥ 0,75	1,00 pada data sintetis; tidak dapat dibandingkan dengan target
Akurasi top-3 RCA	≥ 0,70	Belum dilaporkan pada percobaan A0–A3
Pelanggaran kebijakan yang lolos	0	Nol rilis penuh tanpa persetujuan; uji injeksi 20 pelanggaran belum dilaporkan
Tingkat kegagalan perubahan dan tingkat kerja ulang	≤ A0	Dihasilkan harness (simulasi); laju *rollback* A3 7,4% dibandingkan A0 5,6%
Tingkat *override* manusia	≤ 10%	Belum diukur

9.5 Uji Statistik

Karena distribusi waktu penyelesaian umumnya menceng (*skewed*), kami merancang uji non-parametrik pada Tabel 42.

Tabel 42. Uji statistik yang dirancang

Keperluan	Uji atau ukuran
Perbandingan berpasangan (tiket sama, konfigurasi berbeda)	Wilcoxon *signed-rank*

Keperluan	Uji atau ukuran
Perbandingan lebih dari dua konfigurasi	Friedman, dengan uji lanjut Nemenyi
Ukuran efek	Cliff's delta ($\|d\| \geq 0{,}474$ tergolong besar)
Tingkat signifikansi	$\alpha = 0{,}05$ dengan koreksi Holm-Bonferroni

Ukuran efek harus dilaporkan, bukan hanya nilai-p. Perbedaan tiga detik yang signifikan secara statistik tidak bermakna secara praktis apabila garis dasarnya 12 menit.

Hasil yang dijalankan. Uji omnibus Friedman untuk waktu dinding menghasilkan $p = 0{,}0136$ pada lima benih berpasangan, yang berarti waktu berbeda antarkonfigurasi. Namun, nilai absolutnya sangat kecil karena berupa waktu proksi CPU. Uji omnibus untuk waktu CPU dan laju *rollback* tidak signifikan pada $\alpha = 0{,}05$. Kekuatan uji terbatas karena uji berpasangan hanya memakai lima benih. Hasil ini tidak boleh dibaca sebagai bukti manfaat bisnis.

9.6 Ancaman terhadap Validitas

Tabel 43 merangkum ancaman terhadap validitas hasil dan cara kami memitigasinya.

Tabel 43. Ancaman terhadap validitas dan mitigasinya

Jenis	Ancaman	Mitigasi
Internal	Kebocoran data (tiket uji pernah dilihat saat pelatihan)	Pemisahan kronologis dan pencatatan model_version
Internal	Konfigurasi garis dasar sengaja dilemahkan	Garis dasar kata kunci dioptimalkan secara wajar (*stemming* dan sinonim), bukan pencocokan mentah
Eksternal	Dataset *open source* tidak sama dengan tiket perusahaan *e-commerce*; tiket sintetis terlalu mudah, terbukti dari *macro-F1* 1,00	Keterbatasan dinyatakan; tiket sintetis berdomain perlu diperkaya dan dataset publik perlu dipakai
Konstruk	SWE-bench mendekati titik jenuh dan sebagian kasus ujinya bermasalah	Skor SWE-bench tidak dijadikan klaim utama
Konklusi	Lima benih membatasi kekuatan uji statistik; metrik DORA dan token dibangkitkan atau diestimasi	Hasil dilaporkan sebagai diskusi *trade-off*; evaluasi lanjutan memakai data nyata

BAB 10

KESIMPULAN DAN REKOMENDASI

10.1 Ringkasan Manfaat Sistem

Rancangan T2D-MAS memberikan lima manfaat. Perlu dibedakan bahwa manfaat pertama dan kedua bersifat proyeksi rancangan berbasis asumsi dan perhitungan, sedangkan manfaat keempat telah diwujudkan pada purwarupa.

1. **Memangkas pekerjaan berulang yang terukur.** Berdasarkan asumsi pada Subbab 2.3, otomasi triase dan deteksi duplikat semantik berpotensi mengembalikan sekitar 550 jam kerja *engineer* per bulan (setara dengan sekitar Rp82,6 juta). Penyebab utamanya adalah pencocokan berbasis *embedding* menangkap tiket yang lolos dari pencocokan kata kunci. Contohnya, kemiripan kosinus antara “crash saat login” dan “login gagal, force-close” mencapai 0,994. Angka penghematan ini masih berupa asumsi dan belum diukur pada data operasional.
2. **Mengubah ekonomi investigasi insiden.** Pemindahan komputasi ke lokasi data melalui agen bergerak menurunkan byte yang melintasi jaringan sekitar 98,97 persen dan mempercepat fase pengumpulan bukti hingga sekitar 24 kali pada skenario enam simpul. Manfaat ini bersifat struktural, bukan sekadar optimasi konstanta. Namun, percobaan A0–A3 belum mengukurnya, sebagaimana dijelaskan pada Subbab 9.4.
3. **Menjadikan otonomi sebagai variabel yang dikendalikan.** Aturan gerbang (τ dan R_{maks}) membuat tingkat otonomi dapat disetel pada setiap keputusan dan dinaikkan secara bertahap seiring bukti empiris, sejalan dengan peta jalan Era 1 sampai 8.
4. **Menjadikan auditabilitas sebagai fitur produk.** Layar *Decision Trace* dan tabel *audit_log* yang bersifat *append-only* menjawab pertanyaan mengapa sistem mengambil suatu keputusan. Kemampuan ini justru paling langka pada produk agentik komersial. Purwarupa telah mewujudkan catatan audit dan jeda persetujuan.

5. **Menyediakan portofolio metode yang proporsional.** Aturan dipakai untuk kepatuhan, ML untuk fitur tabular, DL untuk semantik teks, RL untuk keputusan berurutan, GNN untuk relasi antarlayanan, dan LLM untuk bahasa serta kode. Setiap metode ditempatkan pada tahap yang sesuai dengan biayanya.

10.2 Keterbatasan

Purwarupa memiliki enam keterbatasan yang perlu disampaikan secara terbuka.

1. Purwarupa berjalan pada skala laboratorium.
2. Percobaan A0–A3 memakai 500 tiket sintetis per konfigurasi, bukan dataset publik atau tiket perusahaan, sehingga skor model seperti *macro-F1* 1,00 tidak menggambarkan kinerja pada tiket nyata.
3. Mobilitas agen disimulasikan melalui kontainer, bukan simpul produksi sesungguhnya, dan kontainer tersebut merupakan *sandbox* kelas praktikum, bukan batas keamanan yang diperkeras untuk kode tidak tepercaya.
4. Model bahasa besar belum terhubung. Karena itu, biaya dan latensi LLM belum terukur, dan jumlah token hanya berupa estimasi.
5. Alur dari tiket sampai *deployment* belum berjalan menyeluruh secara otomatis. Modul RCA, RL, dan *benchmark* masih berada pada jalur yang terpisah, dan metrik DORA serta hasil *deployment* dibangkitkan oleh harness.
6. Angka penghematan pada Subbab 2.3 masih berbasis asumsi dan perlu divalidasi dengan data operasional nyata. Selain itu, uji berpasangan hanya memakai lima benih sehingga kekuatan ujinya terbatas.

10.3 Rekomendasi Pengembangan

Enam arah pengembangan yang kami rekomendasikan tersaji pada Tabel 44. Tiga di antaranya, yaitu *ledger* untuk audit, *adaptive AI*, dan *cloud deployment*, secara eksplisit diminta oleh tugas.

Tabel 44. Rekomendasi pengembangan

Arah	Isi	Prioritas
Blockchain atau *ledger* untuk audit	Menyimpan *hash* entri audit_log pada *ledger append-only* (Hyperledger Fabric atau *transparency log* bergaya Sigstore) sehingga jejak keputusan agen tidak dapat dimodifikasi, bahkan oleh administrator. Arah ini relevan bagi industri teregulasi	Sedang
Adaptive AI	Pembelajaran daring dari koreksi manusia pada Approval Inbox. Setiap *override* menjadi label, dan ambang τ disesuaikan otomatis per kategori berdasarkan kalibrasi (diagram keandalan)	Tinggi
Cloud deployment	Menjalankan orkestrator pada Kubernetes dengan afinitas simpul agar agen bergerak dijadwalkan pada simpul yang benar-benar menyimpan data, serta penskalaan otomatis *Worker* berdasarkan panjang antrean	Tinggi
Protokol antaragen terbuka	Mengadopsi MCP untuk integrasi *tool* dan protokol kolaborasi agen terbuka, mengikuti arah platform orkestrasi *enterprise*, agar agen pihak ketiga dapat masuk tanpa mengubah inti sistem	Sedang
Federated learning antarpenyewa	Melatih model triase lintas anak perusahaan tanpa memindahkan tiket, dengan memanfaatkan sifat mobilitas agen yang sudah ada	Rendah
Perluasan ke Era 8	Operasi perusahaan otonom hanya dilakukan setelah tiga syarat terpenuhi: tingkat *override* di bawah 5% selama tiga bulan, nol pelanggaran kebijakan yang lolos, dan tingkat kegagalan perubahan tidak lebih buruk daripada garis dasar	Bertahap

Selain enam arah tersebut, langkah terdekat yang paling bermanfaat adalah menghubungkan agen-agen spesialis secara menyeluruh, lalu mengevaluasinya pada data tiket dan CI publik yang dipisah secara kronologis dengan ukuran mutu tugas yang nyata.

10.4 Penutup

Temuan paling penting dari simulasi pada Subbab 8.5 bukanlah kecepatan agen, melainkan pergeseran letak sumbatan. Setelah triase, investigasi, dan implementasi diotomasi, penundaan terbesar berpindah ke penantian persetujuan manusia. Hal ini menyiratkan bahwa pekerjaan lanjutan yang paling bernilai bukanlah membuat agen lebih pintar, melainkan membuat keputusan manusia lebih cepat dan lebih terinformasi. Caranya adalah dengan ringkasan yang berlandaskan bukti, penyajian risiko yang jujur, dan ambang otonomi yang dikalibrasi dengan data.

Hasil percobaan pada Bab 9 sejalan dengan kesimpulan tersebut. Koordinasi dan mobilitas menambah beban kerja pada konfigurasi yang diuji,

sedangkan nilai kelokalan data bergantung pada volume dan kerahasiaan data yang tidak dikuantifikasi oleh *benchmark* ini. Dengan demikian, T2D-MAS pada tahap ini merupakan purwarupa arsitektur yang berguna, bukan sistem pengiriman otonom. Sistem multiagen yang baik bukan yang meniadakan manusia, melainkan yang menempatkan perhatian manusia tepat pada titik yang paling membutuhkannya.

REFERENSI

[1] J. He, L. Xu, M. Yan, X. Xia, and Y. Lei, “Duplicate Bug Report Detection Using Dual-Channel Convolutional Neural Networks,” in *Proc. 28th Int. Conf. Program Comprehension (ICPC)*, 2020.

[2] H. Isotani et al., “Sentence Embedding and Fine-Tuning to Automatically Identify Duplicate Bugs,” *Frontiers in Computer Science*, vol. 4, art. 1032452, 2023.

[3] W. Y. Ramay, Q. Umer, X. C. Yin, C. Zhu, and I. Illahi, “Deep Neural Network-Based Severity Prediction of Bug Reports,” *IEEE Access*, vol. 7, pp. 46846–46857, 2019.

[4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019.

[5] Rahman et al., “Towards Understanding the Impacts of Textual Dissimilarity on Duplicate Bug Report Detection,” arXiv:2212.09976.

[6] S. Hong, X. Zheng, J. P. Chen, Y. Cheng, et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in *ICLR*, 2024.

[7] C. Qian, W. Liu, H. Liu, N. Chen, et al., “ChatDev: Communicative Agents for Software Development,” in *Proc. ACL*, 2024, pp. 15174–15186.

[8] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” arXiv:2405.15793, 2024.

[9] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” arXiv:2310.06770, 2023.

[10] D. B. Lange and M. Oshima, “Seven Good Reasons for Mobile Agents,” *Communications of the ACM*, vol. 42, no. 3, pp. 88–89, 1999.

[11] D. B. Lange and M. Oshima, *Programming and Deploying Java Mobile Agents with Aglets*. Reading, MA: Addison-Wesley, 1998.

[12] FIPA, *FIPA Contract Net Interaction Protocol Specification*, SC00029H, Foundation for Intelligent Physical Agents, 2002.

[13] H. Spieker, A. Gotlieb, D. Marijan, and M. Mossige, “Reinforcement Learning for Automatic Test Case Prioritization and Selection in

Continuous Integration," in *Proc. 26th ACM SIGSOFT ISSTA*, 2017, pp. 12–22.

[14] M. Bagherzadeh, N. Kahani, and L. Briand, "Reinforcement Learning for Test Case Prioritization," arXiv:2011.01834, 2021.

[15] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *ICLR*, 2017.

[16] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs (GraphSAGE)," in *NeurIPS*, 2017.

[17] "A Comprehensive Survey on Root Cause Analysis in (Micro) Services: Methodologies, Challenges, and Trends," arXiv:2408.00803, 2024.

[18] S. Zhang et al., "DiagFusion: Robust Failure Diagnosis of Microservice Systems through Multimodal Data," 2023.

[19] A. Lamkanfi, J. Pérez, and S. Demeyer, "The Eclipse and Mozilla Defect Tracking Dataset: A Genuine Dataset for Mining Bug Information," in *Proc. 10th Working Conf. Mining Software Repositories (MSR)*, 2013, pp. 203–206.

[20] "GitBugs: Bug Reports for Duplicate Detection, Retrieval Augmented Generation, Triage, and More," arXiv:2504.09651, 2025.

[21] LogPAI Team, "bughub: A Collection of Free-Text Bug Reports for Duplicate Issue Identification," GitHub repository. [Online]. Available: https://github.com/logpai/bughub

[22] M. Rath and P. Mäder, "The SEOSS 33 Dataset — Requirements, Bug Reports, Code History, and Trace Links for Entire Projects," *Data in Brief*, vol. 25, 2019.

[23] Atlassian, "AI Feature Guide — Jira Service Management" dan "Agents | Rovo," dokumentasi produk. [Online]. Available: https://www.atlassian.com/software/rovo

[24] ServiceNow, "AI Agents," halaman produk. [Online]. Available: https://www.servicenow.com/products/ai-agents.html

[25] IBM, "watsonx Orchestrate," halaman produk. [Online]. Available: https://ibm.com/watsonx-orchestrate

[26] GitHub, "Copilot coding agent is now generally available," *GitHub Changelog*, 25 Sep. 2025. [Online]. Available: https://github.blog/changelog/2025-09-25-copilot-coding-agent-is-now-generally-available/

[27] Google Cloud / DORA, *State of AI-Assisted Software Development*, 2025.

Catatan: rujukan [23] sampai [27] merupakan sumber web yang cepat berubah. Tanggal akses belum dicantumkan dan perlu dilengkapi dengan format "diakses pada tanggal … 2026" sebelum pengumpulan.

TAUTAN KODE PROGRAM

Tabel 45 memuat artefak proyek beserta tautannya. Alamat tautan belum tersedia pada dokumen sumber sehingga kolom tautan dibiarkan kosong dan perlu diisi oleh ketua kelompok sebelum pengumpulan.

Tabel 45. Artefak dan tautan proyek

Artefak	Tautan	Keterangan
Repositori utama	https://github.com/zidanarsyad/t2d-mas	*Monorepo* berisi agents/, data/, rca/, envs/, eval/, dan frontend/
Video demonstrasi (5 menit)	On going	Alur: tiket masuk, triase, migrasi agen, PR, lalu *deploy*
Slide presentasi	https://docs.google.com/presentation/d/15sjg7T5GHyDjjVaX6ooKesgJbbyGopLR/edit?usp=sharing&ouid=103686854163805175887&rtpof=true&sd=true	Terdapat 14 slide

12.1 Struktur Repositori

Struktur repositori yang kami sarankan tersaji berikut ini.

```
t2d-mas/
├── README.md                  # cara menjalankan dalam 5 perintah
├── docker-compose.yml         # orkestrator + 6 node + redis + postgres
├── agents/                    # base, bus, broker_assign, mobility,
security_policy, orchestrator
├── triage/                    # embedder, dedup, severity, gate
├── rca/                       # graph_builder, model, explain, baseline
├── envs/                      # test_prioritization, canary_deploy
├── data/                      # schema.sql, loaders, preprocess, mock_api
├── eval/                      # runner, metrics, security_injection, stats,
report
├── frontend/                  # React dashboard
├── assets/                    # gambar1..gambar4 untuk laporan
└── results/                   # keluaran eksperimen bertanggal
```

LAMPIRAN A

STATUS KELENGKAPAN DOKUMEN

Lampiran ini mencatat bagian laporan yang masih memerlukan pelengkapan oleh tim sebelum pengumpulan. Tabel 46 memuat setiap butir beserta penanggung jawabnya.

Tabel 46. Butir yang perlu dilengkapi

No.	Lokasi	Butir	Penanggung jawab
1	Sampul	Nama dosen pengampu dan program studi	*Leader*
2	Bab 1	Nama kelompok dan kelas telah tercantum; NIM diverifikasi ulang	*Leader*
3	Bab 2.3	Asumsi diganti dengan data nyata bila tersedia	*Researcher*
4	Bab 4.4 dan 5.3	Jumlah instansiasi agen tidak konsisten (sembilan pada Bab 4.4 dan sebelas pada Bab 5.3 dan 8.1); perlu diseragamkan	*Leader* dan *Designer*
5	Bab 7.3	Skrip pembangkit D7 dan dataset Kaggle D8 (opsional)	*Programmer*
6	Bab 7.3.2	Statistik dataset publik	*Programmer*
7	Bab 8.5	Garis dasar MTTR manual	*Evaluator*
8	Bab 9.4	Metrik yang belum diukur (Tabel 41)	*Evaluator*
9	Bab 11	Tanggal akses sumber web [23]–[27]	*Researcher*
10	Bab 12	Seluruh tautan	*Leader*

LAMPIRAN B

SPESIFIKASI IMPLEMENTASI

Lampiran ini memuat tujuh spesifikasi implementasi yang dipakai untuk membangun modul purwarupa dengan bantuan agen pengodean. Spesifikasi dieksekusi setelah Bab 5 sampai Bab 7 disepakati. Urutan eksekusi mengikuti ketergantungan antarmodul, seperti tersaji pada Tabel 47.

Tabel 47. Urutan eksekusi spesifikasi implementasi

Urutan	Spesifikasi	Letak	Keluaran	Bergantung pada
1	Spesifikasi 4: lapisan data	Bab 7	data/, mock_api/, skema basis data	-
2	Spesifikasi 1: modul triase	B.1	triage/	1
3	Spesifikasi 3: RCA berbasis graf	B.3	rca/	1
4	Spesifikasi 2: lingkungan RL	B.2	envs/	1
5	Spesifikasi 5: kerangka agen dan orkestrator	B.5	agents/, docker-compose	1 sampai 4
6	Spesifikasi 6: dasbor	B.6	frontend/	5
7	Spesifikasi 7: *harness* evaluasi	B.7	eval/, tabel hasil Bab 9	5 dan 6

Sebelum spesifikasi mana pun, kami mengirim satu pesan pembuka sesi berikut.

```
Konteks proyek: saya membangun T2D-MAS, sistem multi-agen untuk pipeline
ticket-to-deployment dengan empat arketipe agen (Scout, Broker, Worker,
Security),
sebagian bersifat mobile (kode berpindah ke node data). Stack: Python 3.11,
FastAPI,
Redis Streams, PostgreSQL + pgvector/FAISS, PyTorch, PyTorch Geometric,
stable-baselines3, React + Tailwind. Seluruh keputusan berisiko melewati
aturan gerbang
Aksi = Otonom jika P >= tau DAN risk <= R_max, selain itu eskalasi ke
manusia.
Prinsip yang tidak boleh dilanggar:
(1) tidak ada merge ke main atau full deploy tanpa flag human_approved,
(2) setiap aksi agen menulis ke audit_log yang append-only,
(3) data mentah tidak boleh meninggalkan node produksi — hanya fitur
teragregasi,
(4) semua angka pada laporan harus dapat direproduksi dari kode dengan seed
tetap.
Balas "siap" lalu tunggu instruksi modul berikutnya.
```

B.1 Modul Triase (Deduplikasi dan Severity)

```
Buat modul Python `triage/` untuk sistem multi-agen dengan spesifikasi:
1) `embedder.py`: bungkus sentence-transformers ("all-MiniLM-L6-v2" sebagai
default,
```

```
   dapat diganti via config). Fungsi encode(texts: list[str]) -> np.ndarray
ternormalisasi L2.
2) `dedup.py`: kelas DuplicateDetector dengan FAISS IndexFlatIP.
   - add(ticket_id, text), search(text, k=10) -> [(ticket_id, cosine_score)]
   - is_duplicate(text, tau=0.85) -> (bool, best_match_id, score)
   - Tiga zona keputusan: score>=tau -> auto-link; 0.70<=score<tau ->
eskalasi manusia;
     score<0.70 -> tiket baru.
3) `severity.py`: klasifikator 4 kelas (Low/Medium/High/Critical).
   - Baseline: TF-IDF + LinearSVC dengan kalibrasi Platt agar keluar
probabilitas.
   - Utama: fine-tune DistilBERT (HuggingFace Trainer), keluaran softmax.
   - Fungsi predict(text) -> dict kelas->probabilitas, plus predicted_class
dan confidence.
4) `gate.py`: implement aturan Aksi = Otonom jika P>=tau dan risk(k)<=R_max,
selain itu eskalasi.
   Tabel risk: Low=0.1, Medium=0.3, High=0.6, Critical=1.0. R_max default
0.6.
5) `tests/`: pytest yang memverifikasi contoh numerik dari laporan:
   cos(A,B)=0.994 dan cos(A,C)=0.119 dengan toleransi 1e-3, serta softmax
   [1.2,0.5,2.1,0.3] menghasilkan P(High)=0.564 +/- 0.001.
Sertakan type hints, docstring, dan requirements.txt. Jangan memakai API
berbayar.
```

B.2 Lingkungan RL

```
Implementasikan dua environment Gymnasium untuk sistem multi-agen CI/CD:
1) `envs/test_prioritization.py` (gaya Retecs):
   - Observasi per tes: [durasi_ternormalisasi,
siklus_sejak_eksekusi_terakhir,
     rasio_kegagalan_historis, flag_flaky, skor_kedekatan_dengan_diff]
   - Aksi: skor prioritas kontinu per tes, dipotong pada anggaran waktu T
   - Reward: NAPFD (Normalized Average Percentage of Faults Detected)
dikurangi
     lambda * total_waktu_eksekusi
   - Data: hasilkan simulasi 200 siklus CI dari distribusi yang dapat
direproduksi (seed),
     dan sediakan loader untuk dataset CI publik bila tersedia.
2) `envs/canary_deploy.py`:
   - State: [risk_score, ukuran_diff, rasio_lulus_qa, jam_hari,
jumlah_rollback_30hari]
   - Aksi diskret: hold / canary_5 / canary_25 / full
   - Reward: +1 tanpa pelanggaran SLO; -5 rollback; -0.2 per jam tunda
   - Terminasi: setelah 30 menit simulasi pasca-deploy
3) Latih dengan stable-baselines3 (PPO dan DQN), simpan kurva pembelajaran,
   bandingkan terhadap dua baseline: urutan acak dan heuristik
"gagal-terbaru-duluan".
```

B.3 Modul RCA Berbasis Graf

```
Bangun `rca/` dengan PyTorch Geometric:
1) `graph_builder.py`: bentuk graf berarah dari data jejak (trace) layanan.
   Node = service instance, fitur = [latency_z, error_rate_z, cpu_z, mem_z,
saturation_z]
   dihitung terhadap baseline 7 hari. Edge = pemanggilan, bobot = volume
request.
2) `model.py`: GraphSAGE 2 lapis + readout per-node -> skor kecurigaan.
   Latih sebagai ranking (margin ranking loss) memakai label insiden
historis.
3) `explain.py`: pakai GNNExplainer untuk mengeluarkan subgraf penjelas top-3
node.
4) `baseline.py`: implementasi pembanding PageRank/random-walk pada graf
dependensi.
5) Verifikasi unit: pada graf contoh A->B->C dengan fitur
   A=[2.1,1.0], B=[2.6,1.2], C=[3.4,2.8], W=0.5I, w=[0.6,0.4],
   skor harus 1.850 / 2.225 / 2.600 (toleransi 1e-3).
```

```
6) Evaluasi: Top-1 dan Top-3 accuracy, Mean Reciprocal Rank. Gunakan dataset
publik
   microservice (mis. rilis data insiden AIOps) atau generator insiden
sintetis
   yang menyuntikkan kegagalan pada node tertentu.
```

B.4 Lapisan Data

```
Buat paket `data/` untuk proyek multi-agen:
1) `schema.sql`: DDL PostgreSQL untuk tabel tickets, ticket_embeddings,
agents, messages,
   evidence, tasks, code_changes, test_runs, deployments, policies,
audit_log, slo_baselines
   sesuai deskripsi di Bab 7.1. audit_log harus append-only (trigger menolak
UPDATE/DELETE).
2) `loaders/gitbugs.py` dan `loaders/eclipse_mozilla.py`: unduh (atau baca
dari path lokal),
   normalisasi ke skema tunggal {ticket_id, title, body, component, severity,
created_at,
   duplicate_of}, dan tulis ke Postgres + Parquet.
3) `preprocess.py`: pembersihan stack trace berulang, normalisasi versi,
masking PII
   (regex email/telepon/token), dan pembagian kronologis train/val/test
70/10/20.
4) `mock_api/`: FastAPI yang meniru Jira, GitHub, observability, CI, dan
infra sesuai
   tabel Bab 7.4, dengan latensi acak 50-300 ms dan tingkat error 2% agar
agen
   terlatih menangani kegagalan.
5) `report_dataset_stats.py`: cetak jumlah tiket, rasio duplikat, distribusi
severity,
   dan panjang deskripsi (median, p95) ke CSV dan grafik, untuk mengisi Tabel
31.
Wajib: idempoten, dapat dijalankan ulang, dan mencatat dataset_version di
setiap keluaran.
```

B.5 Kerangka Agen dan Orkestrator

```
Bangun `agents/` dengan arsitektur berikut:
1) `base.py`: kelas abstrak Agent dengan lima komponen kanonik — perceive(),
remember(),
   reason(), act(), report(). Setiap agen punya atribut role, mobility
(STATIC|MOBILE),
   autonomy_policy, dan menulis ke audit_log pada setiap aksi.
2) `bus.py`: message bus di atas Redis Streams. Pesan mengikuti skema
FIPA-ACL pada
   Bab 5.6.2 (performative, correlation_id, content, policy_context,
signature).
   Sediakan consumer group agar beberapa Worker dapat bekerja paralel.
3) `broker_assign.py`: implementasi Contract Net Protocol — cfp, kumpulkan
propose
   sampai deadline, pilih utilitas tertinggi U = 0.5*skill + 0.3*(1-load) +
0.2*(1-cost),
   kirim accept/reject. Uji unit harus mereproduksi contoh W1=0.63, W2=0.69,
W3=0.66.
4) `mobility.py`: MobileAgentRuntime yang mengemas kode+state menjadi bundle,
   mengirim ke kontainer target lewat API runtime, mengeksekusi di sandbox
dengan
   batas cgroup, lalu menarik kembali hasil. Catat bytes_sent dan
bytes_returned.
   Implementasikan aturan migrasi: migrasi jika L_node > (A+R)*3.
5) `security_policy.py`: verifikasi tanda tangan ed25519 bundle agen, egress
allowlist
   field hasil, secret scanning, dan evaluasi gate rule (tau, risk_max).
6) `orchestrator.py`: state machine 9 tahap dengan tiga human checkpoint.
   Bila gerbang mengeskalasi, tiket masuk antrean approval dan pipeline TIDAK
berlanjut.
```

```
7) `docker-compose.yml`: 1 orkestrator + 6 kontainer node + redis + postgres,
   sehingga migrasi agen dapat didemokan secara nyata.
Wajib: logging terstruktur JSON dengan correlation_id, dan skenario uji
end-to-end
yang mereproduksi tabel simulasi flash sale pada Bab 8.5.
```

B.6 Dasbor

```
Buat frontend React + Tailwind + Recharts dengan lima layar:
1) Pipeline Overview: kanban 9 kolom, kartu tiket menampilkan ID, severity
berwarna,
   agen pemegang, waktu di tahap, dan badge "menunggu manusia".
2) Agent Status: daftar agen dengan ikon statis/mobile, host, status, beban,
heartbeat;
   untuk agen mobile tampilkan jalur migrasi dan byte yang berpindah.
3) Decision Trace: linimasa per tiket berisi agen, input ringkas, keluaran,
confidence,
   kebijakan (tau, risk_max), hasil gerbang, dan penyetuju. Baris dapat
diperluas
   untuk melihat bukti mentah.
4) Approval Inbox: antrean eskalasi terurut risiko x waktu tunggu, dengan
ringkasan
   yang menyertakan tautan bukti, tombol Setujui/Tolak, dan wajib alasan saat
menolak.
5) Performance Report: lima metrik DORA + metrik agen (akurasi triase, recall
duplikat,
   biaya token, byte jaringan, rasio override), grafik garis 30 hari, tombol
ekspor CSV.
Teknis: konsumsi FastAPI backend lewat server-sent events untuk pembaruan
real-time.
Sediakan mode "Static vs Mobile" berupa toggle yang menampilkan perbandingan
waktu dan byte pada tiket yang sama. Aksesibilitas: jangan gunakan warna
sebagai
satu-satunya pembawa makna; sertakan label teks. Buat dalam mode gelap dan
terang.
```

B.7 Harness Evaluasi

```
Buat `eval/` yang menjalankan eksperimen empat arm:
1) `runner.py`: jalankan arm A0 (rule-based), A1 (single-agent), A2 (MAS
statis),
   A3 (MAS + mobile) pada himpunan tiket identik, 5 seed, catat per tiket:
   waktu dinding per tahap, byte jaringan (hitung dari instrumentasi bus dan
   MobileAgentRuntime), jumlah pesan, token LLM, CPU-detik, puncak memori.
2) `metrics.py`: implementasi Recall@k, MAP, macro-F1, NAPFD, Top-k accuracy
RCA,
   serta lima metrik DORA (deployment frequency, lead time, failed deployment
   recovery time, change failure rate, rework rate).
3) `security_injection.py`: suntikkan 20 pelanggaran kebijakan sintetis
   (rahasia di diff, deploy saat change freeze, PII pada egress, severity
Critical
   tanpa persetujuan) dan hitung berapa yang lolos ke tahap berikutnya.
Target 0.
4) `stats.py`: Wilcoxon signed-rank, Friedman + Nemenyi, Cliff's delta,
   koreksi Holm-Bonferroni.
5) `report.py`: hasilkan tabel Markdown siap tempel ke Bab 9.4 dan grafik
   (boxplot waktu per arm, bar chart byte jaringan A2 vs A3, kurva ROC
severity).
Wajib: seluruh hasil ditulis ke results/<timestamp>/ bersama file
konfigurasi,
dan dapat direproduksi dengan satu perintah `make eval`.
```